

Assignment 3 Design Rationale

Group E

Suchir Ganesh Jaiganesh (35247096)

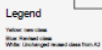
Chathya Attanayake (35427965)

Jewell Eudocia Gomes (35627239)

Victoria Tay Wen Xie (35181257)

Aida Ab Aziz (34597506)

UML DIAGRAM



DESIGN RATIONALE

Things I had changed from the feedback for A2: I had made BaseTeleportStrategy an abstract class to act as a template for the teleport strategies and had removed the previous TeleportStrategy interface as it will be redundant code for they work in similar ways. It now also has shared utilities like findRandomValidLocation() so that other strategies like TeleportTubeStrategy and AlienCubeStrategy can use it which complies to DRY principle. It now also upholds the KISS for every concrete strategy will just extend from it.

A. Implementation of QuotaManager

Design 1: Static Fields on a global class

Store all quota state, accumulated credits, rank, remaining turns and super computer facility access flag as static fields on a utility class which can be accessed directly when needed.

Pros	Cons
No constructor injection required as any class can access quota state with a direct static call.	Shared state makes unit testing difficult.
Simple to implement quickly as no need to reference through EclipseNebula and other constructors.	Violates Dependency Inversion Principle because classes depend directly on a concrete implementation, creating tight coupling.

Design 2: Injected QuotaManager instance (chosen)

A single QuotaManager is created in EclipseNebula and passed to the dependent classes

Pros	Cons
Centralises quota logic in one class, satisfying SRP.	Requires constructor wiring which is complex as it introduces connascence of construction order
Easy to test with independent instances.	Corporate termination performs a map scan for the location of SuperComputer tile per quota cycle.
Follows Open/Closed Principle as changing the scaling rate need to change one method, no consumer classes is touched.	

Justification for choosing Design 2: The testability and SRP gains outweigh the wiring complexity introduced by constructor injection. With static fields, QuotaManagerTest cannot instantiate a fresh isolated instance per test case, meaning one test's state bleeds into the next. Now making automated verification of the deadline cycle completely unreliable. An injected instance solves this cleanly while keeping each class responsible for only its own concern.

B. Economic Capability Interfaces on depositable, sellable and purchasable

Design 1: Single interface

One interface that declares getter for selling price, deposit value and purchase price and every item just implements them.

Pros	Cons
Fewer classes as one interface instead of 3 interfaces.	Violates Interface Segregation Principle as for example AluminiumScrap cannot be sold, yet it would be forced to implement <code>getSellPrice()</code> with a meaningless stub, introducing dead code
SuperComputer need to check only one capability per item, simplifying the <code>allowableActions</code> loop	Creating tight coupling between unrelated economic behaviours.
	If the items are added with new economic behaviour, it requires modifying the interface and every implementing class, violating OCP.

Design 2: Separate Depositable, Sellable, and Purchasable Interfaces (chosen)

Depositable, Sellable, and Purchasable are separate interfaces. Items implement only what applies to them.

Pros	Cons
Interface Segregation Principle as each interface is minimal. No item is forced to implement an irrelevant method. For example, AluminiumScrap cannot be sold so it will not force it to give sell price.	More interfaces and action classes. For items that are both sellable and depositable, the player sees two separate menu entries, increasing action-list length
It is open for extension as a new item can implement whichever interfaces that suit to it.	Additional capability checks are required.
KISS and SRP satisfied as each action class has exactly one responsibility and depends on exactly one interface for each action class.	

Justification for choosing Design 2 is that ISP is an important principle as if design 1 is chosen, it will force items like AluminiumScrap to implement `getSellPrice()` which it cannot be sellable is a clear design smell. Design 2 is opened for extension which means that new item

can just implements whichever interfaces that suit to them without having dead code if they only are just depositable for example.

C. Cut Action and Cuttable Interface

Design 1: instanceof-Based Logic

CutAction identifies the cuttable objects using conditional checks

Pros	Cons
Simple to implement initially as all cutting logic is in one place	Every new cuttable type requires adding another instanceof branch to the CutAction class so it violates OCP.
No additional interface required; the action is self-contained.	SRP is also violated as CutAction becomes responsible for the side effects for each cuttable it cuts
	CutAction is coupled to every concrete cuttable class by name which introduces connascent of type.

Design 2: Cuttable Interface (chosen)

Each cuttable object implements its own cutting behaviour

Pros	Cons
New cuttable types requires no changes to CutAction so following OCP.	Inventory items require a null location parameter.
Removes type coupling through polymorphism.	canBeCut()'s default method relies on HAS_PLASMA_CUTTER being correctly set on PlasmaCutter
SRP as each cuttable class handles its own post-cut consequences for example, Vent becomes floor and drops an IndustrialFan.	

Justification for choosing Design 2: The Cuttable interface declares canBeCut() and executeCut(), allowing each cuttable class like Vent and AlienCube to provide its own specific cutting behaviour — this is polymorphism applied correctly. The instanceof alternative requires CutAction to be reopened and modified every time a new cuttable object is introduced, which directly violates OCP and makes CutAction increasingly fragile and harder to maintain over time.

D. Quota Failure on facility lockout and super computer facility access termination

Design 1: Checks in every action

Each action class (SellAction, DepositAction, PurchaseAction) independently checks whether facility access is still active before executing, and suppresses itself if the quota has failed.

Pros	Cons
Each action is self-contained and does not depend on SuperComputer.	Repeated logic on isFacilityAccessActive() check violates DRY.
Different actions could be disabled individually, flexibility.	Violated SRP as each action classes become responsible for both the transaction logic and enforcing global quota policy.

Design 2: Central guard in SuperComputer (chosen)

SuperComputer.allowableActions() will block all actions when the access is revoked.

Pros	Cons
KISS as one boolean check at the single point of action generation and no scattered guards across multiple classes	Players may not receive explicit lockout feedback.
SRP as now SuperComputer just decide which actions to offer and QuotaManager decides when to revoke access	

Justification for choosing Design 2: The distributed check design is a DRY and SRP violation that invites silent maintenance bugs as the codebase grows. If a new action class is added and the developer forgets the isFacilityAccessActive() check, facility access is only partially revoked with no compile-time warning. A single guard at the top of SuperComputer.allowableActions() means every current and future action inherits the lockout automatically, with zero risk of omission.

E. PlasmaCutter: Purchase Side Effects and Cut Discovery

Design 1: Hardcode Side Effects Inside PurchaseAction

PurchaseAction detects that the purchased item is a PlasmaCutter through usage of instanceof and applies BurningStatus directly, keeping all purchase consequence logic in the action class.

Pros	Cons
No additional interface method required as PurchaseAction is the single place to read when tracing what happens on any purchase.	Violates OCP as every item with a unique purchase side effect requires modifying PurchaseAction with another instanceof branch
It is simple to implement for a single item with special behaviour.	Violates SRP as PurchaseAction becomes responsible for both the transaction mechanics and the consequences of every purchasable item

Design 2: purchasedBy() and failedPurchaseBy() on the Purchasable Interface (chosen)

PlasmaCutter implements Purchasable and overrides purchasedBy() to apply BurningStatus on purchase, and failedPurchaseBy() for a domain-specific error message. PurchaseAction calls purchasedBy() generically not knowing of what effect it will incur.

Pros	Cons
Follows OCP as any future purchasable item implements its own purchasedBy() in PurchaseAction never changes regardless of how many new items are added.	allowableActions(Location) casts ground to Cuttable after an ability check, then this cast would throw ClassCastException if a Ground incorrectly declared the CUTTABLE ability without implementing the Cuttable interface
SRP as PlasmaCutter owns the consequence of its own purchase; PurchaseAction owns the transaction mechanics only	
PlasmaCutter's allowableActions overrides inject CutActions directly from the item so responsibility is correctly localised	

Justification for choosing Design 2: Placing side-effect logic inside the item itself is the correct application of OCP and SRP. PurchaseAction becomes fully generic — it calls

purchasedBy() without knowing or caring what the effect is. Any future purchasable item with unique consequences simply overrides that method. Design 1 forces PurchaseAction to grow an instance of branch for every new item, permanently coupling the action class to concrete item types.

F. Resource Item Side Effects (Deposit and Sell Consequences)

Design 1: Centralised Effect Handler in DepositAction and SellAction

DepositAction and SellAction contain switch-case or instanceof logic to apply item-specific effects. For example, they can detect AluminiumScrap to apply cut dame or AlienArtifact to trigger teleportation.

Pros	Cons
All deposit and sell consequences are visible in one class, making it easy to trace and change as all logic are centered in one class.	Violates OCP as adding a new depositable item with unique consequences requires modifying DepositAction this means that the class can never be closed for modification.
No depositBy()/soldBy() methods needed on items as the action classes are fully self-contained.	Violates SRP as DepositAction becomes responsible for transaction mechanics, credit delegation, and the domain logic of every item type simultaneously
	Connascence of type: DepositAction is coupled by class name to every resource item, creating fragile dependencies.

Design 2: Side Effects Owned by Each Item via depositBy() and soldBy() in each depositable or sellable classes (chosen)

Each resource item owns its consequences inside depositBy() and soldBy(). DepositAction removes the item, calls quotaManager.addCompanyCredits(), calls depositable.depositBy(), and returns the combined message. There is no need of conditional logic, no type checks.

Pros	Cons
Allows OCP as DepositAction and SellAction never change when new items are added. A new item simply implements its interface and defines its own effect method.	Minor duplication exists in wallet retrieval code.
Connascence of name only: DepositAction knows only the method name depositBy() — not the item type	
DRY as AlienArtifact.depositBy() reuses BaseTeleportStrategy.findRandomValidLocation() rather than reimplementing random-location search, reusing the pre-existing code	

Justification for choosing Design 2: Centralising consequences in action classes is OCP and SRP violation. Item-owned effects keep each class responsible for its own domain behaviour and keep action classes permanently stable, promoting SRP and also no connascence of type.

Conclusion:

The REQ1 design consistently follows SOLID principle as the responsibilities are delegated to the classes that own the behaviour, usages of interfaces which minimizes coupling and maximise extensibility.

The QuotaManager class centralises all quota cycle state including accumulated credits, company rank, remaining turns, and facility access status within a single object created in EclipseNebula and passed to all dependent classes through constructor injection. This removes reliance on static global state and ensures that all dependent classes work with an injected instance rather than directly accessing a shared global class, improving adherence to the Dependency Inversion Principle. It also consolidates all quota-related behaviour such as rank progression, deadline checks, facility lockout, and corporate termination into one location, so other classes do not need to manage or understand quota logic, supporting the Single Responsibility Principle.

Separating economic capabilities into three distinct interfaces named Depositable, Sellable, and Purchasable ensures that item classes only implement behaviours that are relevant to them. For example, AluminiumScrap is only depositable and does not need to implement selling methods, while IndustrialFan and AlienArtifact support both selling and depositing. This avoids forcing irrelevant method implementations and improves clarity and design correctness, in line with the Interface Segregation Principle. It also allows action classes such as DepositAction, SellAction, and PurchaseAction to depend on a single focused interface each, avoiding conditional logic and ensuring they remain stable as new item types are introduced, supporting the Open-Closed Principle.

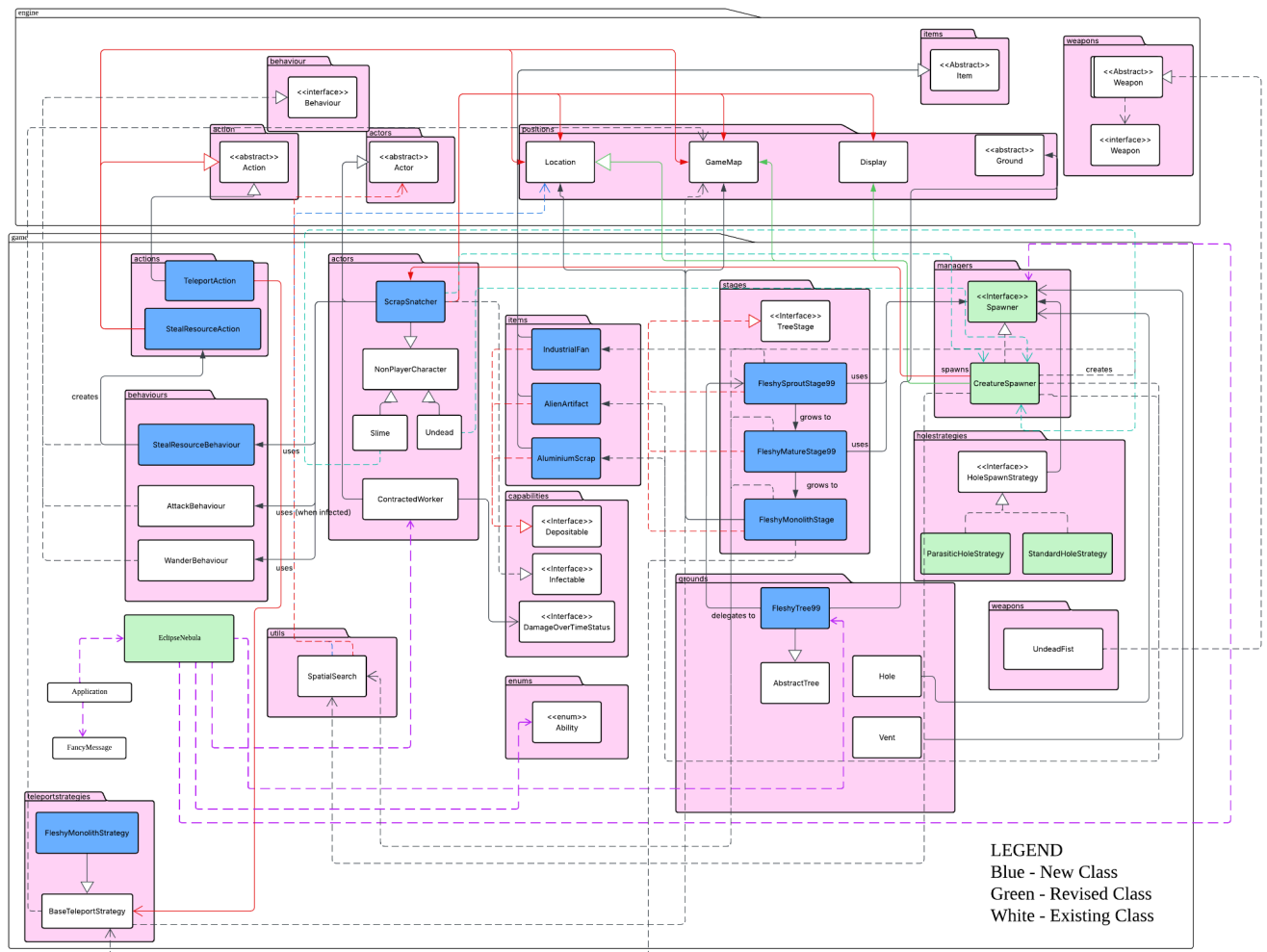
The Cuttable interface defines canBeCut() and executeCut() methods, which are implemented by classes such as Vent and AlienCube. CutAction simply calls executeCut() without needing to know the specific type of object being cut. This avoids using instanceof checks inside CutAction, which would require modifying the class whenever a new cuttable object is added and is considered a code smell that violates the Open-Closed Principle. Instead, each class defines its own cutting behaviour, such as Vent converting into a floor tile and dropping an IndustrialFan, or AlienCube producing an AlienArtifact and applying poison. This ensures each class is responsible for its own behaviour, aligning with the Single Responsibility Principle. The default canBeCut() method also standardises the check for whether a player has a plasma cutter, avoiding duplicated logic across classes.

The purchase side effect for PlasmaCutter is defined inside purchasedBy() on the item itself rather than hardcoded inside PurchaseAction. This means PurchaseAction is fully generic so now it calls purchasedBy() without knowing what the effect is. For any future purchasable item with unique consequences simply overrides that method without requiring any change

to PurchaseAction (Open-Closed Principle). Each item class is responsible for what happens when it is purchased, sold, or deposited, rather than delegating that knowledge to the action class (Single Responsibility Principle).

Finally, the facility lockout on quota failure is handled by a single guard clause at the top of SuperComputer's allowableActions() method, checking isFacilityAccessActive() before returning any actions. The alternative of placing that check inside every action class individually would duplicate the same logic across SellAction, DepositAction, and PurchaseAction, creating a maintenance hazard where a future action class could silently omit the check and provide partial access. By placing the guard at the single point of action generation, the lockout is inherited automatically by every current and future action offered by the Supercomputer, with no repeated logic and no risk of omission (DRY, Single Responsibility Principle).

UML DIAGRAM



DESIGN RATIONALE

A. Scrap Snatcher Hoarding and Stealing Mechanics

This section explains how the Scrap Snatcher detects and picks up depositable items from the ground. The Snatcher needs to identify which items on the ground can be deposited to the Supercomputer (Aluminium Scrap, Industrial Fan, Alien Artifact) and then "snatch" them into its inventory.

Design 1: Implement stealing logic directly inside ScrapSnatcher.playTurn()

Pros	Cons
Simple implementation with no extra classes	Violates SRP: ScrapSnatcher would handle both creature behavior and item detection
Direct logic flow easy to trace	High coupling: Cannot reuse stealing logic for other creatures
Fewer files to maintain	Violates OCP: Adding new stealable items requires modifying ScrapSnatcher

Design 2: Separate Behaviour and Action classes (CHOSEN)

Pros	Cons
SRP: StealResourceBehaviour handles detection, StealResourceAction handles execution	Requires two additional classes
DRY: Stealing logic can be reused for other creatures	Indirect logic flow may be harder to trace initially
OCP: New stealable items added via Depositable interface, no changes to behaviour/action	
DIP: Depends on Depositable abstraction, not concrete items	

Justification: Design 2 was chosen because REQ2 requires the Snatcher to steal "any resources that can be deposited to the supercomputer" - this implies multiple item types. By having StealResourceBehaviour check for Depositable items via asCapability(), the stealing system automatically works with any future depositable item without modification (OCP). The separation of behaviour and action follows the established pattern in the codebase (AttackBehaviour + AttackAction), maintaining consistency.

B. Weapon Selection for Infected Snatcher

When the Scrap Snatcher becomes infected by a Parasite, it loses its hoarding ability and starts attacking workers. The requirements specify it attacks with "its bare fist, dealing 1 damage point with a 10% chance to hit."

Design 1: **Create new ScrapSnatcherBareFist class**

Pros	Cons
Custom name specific to ScrapSnatcher	Violates DRY: Duplicates existing UndeadFist functionality
Independent evolution of weapon stats	Increases codebase size unnecessarily
Clear naming for debugging	Both classes would have identical parameters (1 damage, 10% hit rate)

Design 2: **Reuse existing UndeadFist (CHOSEN)**

Pros	Cons
DRY: Single source of truth for bare fist weapon	Name "UndeadFist" may not semantically match ScrapSnatcher
Less code to maintain	Requires passing damage/hit rate parameters
Follows lecturer feedback ("better to just use the undead fist")	
Demonstrates code reuse and reduces class clutter	

Justification: Design 2 was chosen following explicit lecturer feedback. The UndeadFist constructor accepts damage and hitRate parameters, allowing instantiation with `new UndeadFist(1, 10)` which perfectly matches REQ2's requirements (1 damage, 10% hit rate). Creating a separate ScrapSnatcherBareFist would duplicate identical logic and violate DRY. The name mismatch is acceptable as the weapon's behavior is identical - both represent a weak, inaccurate punch.

C. Infection Transformation (Rabid State)

What this design addresses:

When a Parasite infects a Scrap Snatcher, the Snatcher must:

- Gain "Infected" status
- Lose its ability to steal items
- Start attacking workers instead
- Take 1 damage per turn until death

Design 1: **Use InfectionStatus only for damage, track behaviour changes via flags**

Pros	Cons
Simple boolean flag for behaviour change	Behaviour logic scattered across multiple methods
No need to modify behaviour map	Risk of inconsistent state between flag and actual behaviours
	Violates SRP: playTurn() becomes responsible for behaviour decisions

Design 2: **Modify behaviour map dynamically (CHOSEN)**

Pros	Cons
SRP: Behaviour changes encapsulated in reactToInfection()	Requires understanding of behaviour priority system
Consistent with existing NPC architecture (Parasite, Undead)	More complex than simple flag
OCP: New behaviours can be added by changing priority numbers	
Self-documenting: behaviour priorities clearly show what Snatcher can do	

Justification: Design 2 was chosen to maintain consistency with the existing NPC architecture. The NonPlayerCharacter base class provides a behaviours TreeMap where priorities determine which action executes first. By removing StealResourceBehaviour (priority 1) and adding AttackBehaviour (priority 10) when infected, the Snatcher seamlessly transitions from hoarding to attacking. This approach requires no additional state checks in playTurn() - the behaviour map itself encodes the Snatcher's capabilities. The infection damage is handled separately in playTurn() because REQ2 specifies "takes 1 damage per turn until it dies," which is separate from the InfectionStatus damage (also 1 HP per turn from the parent class).

D. Loot Explosion on Spawn

When a Scrap Snatcher spawns onto the map, it triggers a "loot explosion" - one random depositable item appears on every adjacent tile (up to 8 items, one per direction).

Design 1: **Hardcode specific item creation in spawnScrapSnatcher()**

Pros	Cons
Simple and direct	OCP violation: Adding new depositable items requires modifying spawn method
No reflection overhead	Code duplication if similar loot explosion needed elsewhere
Easy to understand and debug	Hardcoded item types limit extensibility

Design 2: **Use reflection with DEPOSITABLE_CLASSES list (CHOSEN)**

Pros	Cons
OCP: New depositable items added by updating list, not modifying spawn method	Reflection may fail if class lacks no-arg constructor (mitigated with fallback)
DRY: Single source of truth for depositable item types	Slightly more complex code
Extensible for future requirements	
Follows "any other resources that are added in the future" requirement exactly	

Justification: Design 2 was chosen because REQ2 explicitly states the loot explosion should work for "any other resources that are added in the future." Using reflection with a DEPOSITABLE_CLASSES list makes the system extensible - adding a new depositable item only requires adding its class to the list. The loot explosion iterates through all 8 adjacent tiles (using getExits()) and spawns one random depositable item on each traversable tile. Per the 29/05 edit, the "empty" check was removed, so items spawn regardless of existing ground items.

E. Fleshy Tree Modifications for 99-Deprecated Map

What this design addresses:

On the 99-deprecated map, the Fleshy Tree behaves differently from the original:

- Sprout ('y') spawns Undead instead of Slime
- No Sapling stage - Sprout grows directly to Mature
- Mature ('Y') spawns Scrap Snatcher instead of Undead
- Mature can grow into a Monolith ('H') that warps workers

Design 1: **Modify existing FleshyTree stages with conditional logic**

Pros	Cons
Reuses existing stage classes	Violates OCP: Adding map-specific behavior requires modifying existing stages
No new classes needed	Introduces fragile coupling between stage logic and map identity
Simpler class structure	Breaks original FleshyTree behavior for 20-overflow map

Design 2: **Create new stage classes specific to 99-deprecated map (CHOSEN)**

Pros	Cons
OCP: Original FleshyTree unchanged	More classes to maintain
Clear separation of map-specific behavior	Some code duplication with original stages
Independent evolution of each map's tree behavior	Requires new ground class FleshyTree99

LSP: FleshyTree99 can replace AbstractTree anywhere	
---	--

Justification: Design 2 was chosen because the 99-deprecated map's tree behavior is fundamentally different: Sprout spawns Undead (not Slime), Mature spawns ScrapSnatcher (not Undead), and there is no Sapling stage. Modifying the existing FleshySproutStage and FleshyMatureStage with conditional logic would violate OCP and introduce fragile coupling. Creating separate stage classes (FleshySproutStage99, FleshyMatureStage99, FleshyMonolithStage) keeps the original FleshyTree behavior intact for the 20-overflow map while allowing independent evolution for the 99-deprecated map.

F. Monolith Teleportation

The Fleshy Monolith is the final stage of the tree on the 99-deprecated map. Instead of spawning enemies, every turn it warps any adjacent worker to a random valid location on the same map.

Design 1: Implement teleport logic directly in FleshyMonolithStage

Pros	Cons
Self-contained, no external dependencies	Violates DRY: Teleportation logic would be duplicated
Fewer classes to maintain	High coupling to GameMap and Location logic
	OCP violation: New teleport behaviors would require modifying stage

Design 2: Reuse existing BaseTeleportStrategy (CHOSEN)

Pros	Cons
DRY: Teleportation flow written once in BaseTeleportStrategy.teleport()	One extra class (FleshyMonolithStrategy)
OCP: New teleport behaviors added via new strategy classes	
Consistent with existing teleportation system (Teleportation Tubes, Alien Cube)	
Low connascence: Stage doesn't need to know how teleportation works	

Justification: Design 2 was chosen to maintain consistency with the existing teleportation system established in REQ2 (Teleportation Tubes) and REQ3 (Warper Tree). The BaseTeleportStrategy abstract class provides a reusable teleport() method that handles source effects, destination determination, actor movement, and destination effects. FleshyMonolithStrategy extends BaseTeleportStrategy and overrides determineActualDestination() to return a random valid location via findRandomValidLocation(). This approach demonstrates excellent code reuse and follows the established pattern in the codebase.

G. Spawn Strategy Integration

The Scrap Snatcher must spawn from Holes on the 99-deprecated map and from Vents on the 20-overflow map. This requires updating the spawn strategies to include Scrap Snatcher in their spawn pools.

Design 1: Add ScrapSnatcher to hole strategies with conditional logic

Pros	Cons
Simple modification of existing classes	Risk of breaking existing spawn distributions
	Hardcoded probabilities difficult to maintain

Design 2: Modify spawn probabilities using integer partitioning (CHOSEN)

Pros	Cons
Clear 33/33/33 distribution using random.nextInt(3)	Requires modifying two strategy classes
No switch statement (uses if-else chain)	Probabilities hardcoded
Maintains existing Strategy Pattern architecture	
OCP: New creatures added by adding else-if branch	

Justification: Design 2 was chosen because the StandardHoleStrategy and ParasiticHoleStrategy already use the Strategy Pattern. The original 50/50 distribution was changed to 33/33/33 using random.nextInt(3) mapped to three outcomes. This avoids switch statements (per feedback) while maintaining clear probability distribution. The Vent class was also updated to use ParasiticHoleStrategy instead of hardcoded 50/50 logic, ensuring consistency across all spawn points on the 20-overflow map.

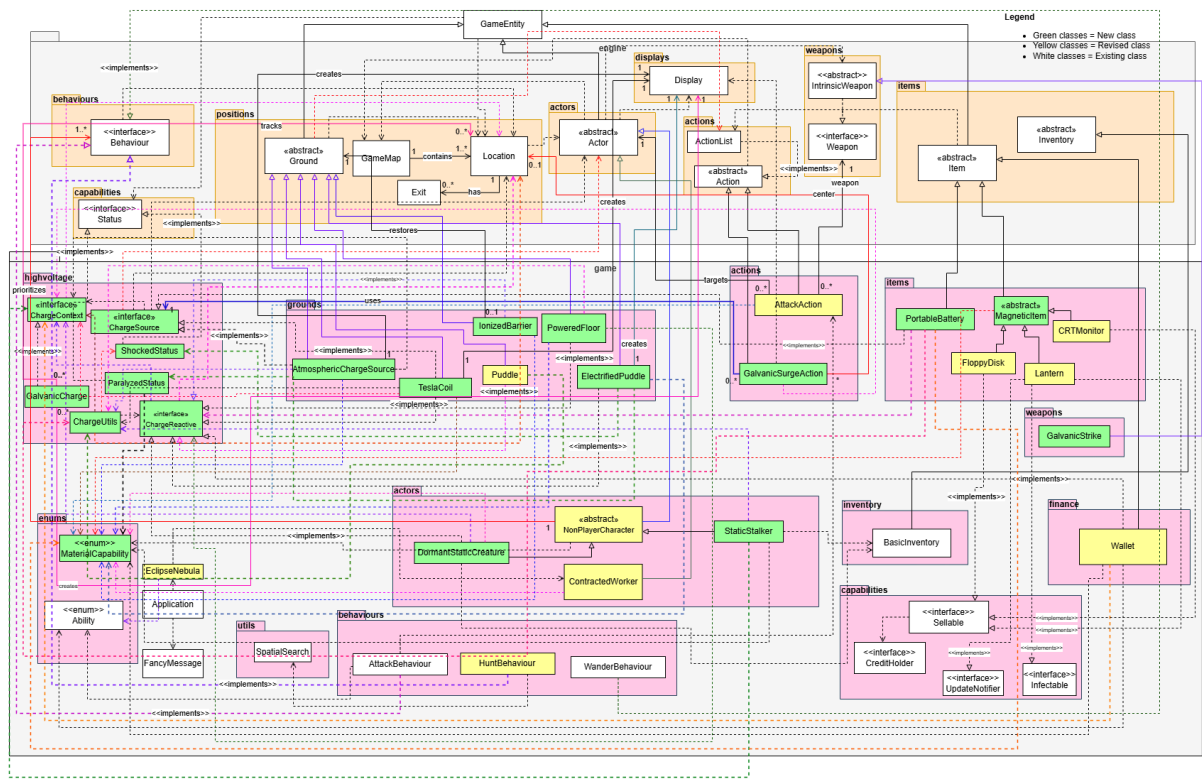
Conclusion

The REQ2 implementation successfully delivers the Scrap Snatcher ecosystem and modified Fleshy Tree behavior for the 99-deprecated map while adhering strictly to SOLID principles. Single Responsibility is demonstrated through separation of concerns: `StealResourceBehaviour` detects depositable items, `StealResourceAction` executes the theft, and each tree stage manages its own growth logic. Open/Closed is satisfied as new depositable items can be added by implementing `Depositable`, and new tree stages by implementing `TreeStage`, without modifying existing code. Liskov Substitution holds because `ScrapSnatcher` extends `NonPlayerCharacter` and `FleshyTree99` extends `AbstractTree`, functioning correctly wherever their parent types are expected. Interface Segregation is evident as `ScrapSnatcher` implements only `Infectable`, not unneeded interfaces, and items implement only `Depositable`. Dependency Inversion is achieved through abstraction: tree stages depend on the `Spawner` interface rather than concrete `CreatureSpawner`, and `StealResourceBehaviour` depends on the `Depositable` interface. The implementation avoids `instanceof` entirely, using capability pattern (`asCapability()`, `hasAbility()`), and uses `display.println()` exclusively for output. No new singletons were introduced; dependencies are injected via constructors. The design is extensible, maintainable, and fully compliant with all REQ2 requirements.

REQ3: CREATIVE MODE

High-Voltage Galvanic System

REQ 3 UML Class Diagram



Design Rationale

REQ 3

A. Architecture of the Electrical Ecosystem

The design introduces three high-level abstractions to decouple energy emitters from reactive resonators, following the Dependency Inversion Principle (DIP).

Design 1: Direct Coupling (Concrete method calls between Emitters and Targets)

Pros	Cons
Simple to implement for a single interaction, like Lightning directly calls Puddle.electrify()	Violates DIP: Emitters must know every possible target type (Grounds, Actors, Items)
Fewer files in the codebase	Violates OCP: Adding a new resonator (e.g., a metallic wall) requires modifying every emitter class
Easier to trace in isolation	High Connascence of Type: Emitters are coupled to the concrete names of resonators
	AtmosphericChargeSource.tick() would require an instanceof Puddle, instanceof

	DormantStaticCreature, and instance of Wallet chain. Every new resonator forces a modification to every emitter.
--	--

Design 2: Interface Driven Abstraction(ChargeSource/ChargeReactive/ChargeContext)

Pros	Cons
Adheres to DIP: Emitters depend only on the ChargeReactive interface, not concrete classes.	Increases the number of files in the game.highvoltage package.
High extensibility (OCP): New emitters (Solar Panels) or resonators (Robot NPCs) can be added without modifying existing electrical logic.	Indirection: Developers must follow interface calls to understand execution flow.
Polymorphic consistency: PortableBattery (Item) can trigger TeslaCoil (Ground) which zaps Wallet (Item) using the same method signature.	Static coupling of ChargeUtils.zapTile() makes unit testing harder without mocking.
ChargeContext as a data carrier decouples surge metadata (damage, source name) from reaction mechanics.	
Eliminates Connascence of Type: ChargeUtils.zapTile() handles damage, ShockedStatus, metamorphosis, and inventory induction without a single instance of check	

Justification: Design 2 was chosen. The design utilizes an interface-driven abstraction (Design 2) involving ChargeSource, ChargeReactive, and ChargeContext to adhere to the Dependency Inversion Principle (DIP). By depending on interfaces rather than concrete classes, emitters like AtmosphericChargeSource remain decoupled from resonators like Puddle or Wallet. This architecture strictly follows the Open-Closed Principle (OCP); new reactive entities (e.g., Conductive Pipes) can be added by implementing ChargeReactive without modifying existing emitter logic. Furthermore, GalvanicSurgeAction demonstrates DIP by accepting a ChargeSource interface, allowing a single Action class to handle both portable Items and stationary Ground objects, satisfying DRY.

B. Recursion Prevention via Visited Set (ChargeContext)

To prevent infinite loops in conductive networks (adjacent PoweredFloor tiles or TeslaCoil-to-TeslaCoil overload chains), the design uses a HashSet<Location> carried within ChargeContext.

Design 1: Simple Counter / Depth-Limit

Pros	Cons
Simple to implement with an integer	Does not prevent back-arcing. For instance,

powerLevel that decrements.	energy can flow A→B→A, causing duplicate damage.
Low memory overhead.	Cannot distinguish revisiting a tile from legitimate new propagation

Design 2: Visited Set (HashSet<Location> in GalvanicCharge)

Pros	Cons
Guarantees a single surge event cannot enter infinite recursion, preventing StackOverflowError.	Each surge creates a temporary HashSet, though negligible for game-scale maps.
Adheres to LSP: Any ChargeContext implementation can be used without resonators knowing how memory is stored.	Connascence of Algorithm: Emitters and resonators must agree on when to call visit(). Mitigated by centralising this in ChargeUtils
Centralized safety check in ChargeUtils.zapTile() ensures consistency.	

Justification: Design 2 was chosen. To prevent infinite loops in conductive networks (adjacent PoweredFloor tiles), the design employs a HashSet<Location> within ChargeContext. Unlike a simple depth-limit, a visited set prevents back-arcing and duplicate damage within a single surge. The visited set check follows a layered responsibility pattern: ChargeUtils.zapTile() handles the global safety guard for entity damage. However, emitters and resonators perform local visit checks to control their specific propagation algorithms before delegating to the utility. For example, TeslaCoil checks charge.visit(loc) before scanning its diamond-shaped AoE to prevent the same tile from being processed twice across overlapping radius calculations. This separation of concerns ensures propagation logic remains with the emitter while safety remains consistent.

C. Centralized Impact Coordination (ChargeUtils)

The design employs a stateless utility class to centralize high-voltage operations (damage application, status effects, inventory induction).

Design 1: Localized Logic (Each class handles its own impact effects)

Pros	Cons
KISS (Keep it Simple, Stupid): All logic for a Puddle is inside Puddle class.	Violates DRY: Applying ShockedStatus and damage would be duplicated across TeslaCoil, AtmosphericChargeSource, and PortableBattery.
Easier to trace single-file behavior.	Inconsistency risk: Changing ShockedStatus duration from 2 to 3 turns requires updating 4 or many files.
	Separating the logic in zapTile() would require every caller to invoke all three in the

	correct order, introducing Connascence of Execution at every call site, which is worse than the complexity contained within one method.
--	---

Design 2: Centralized Utility Class (ChargeUtils)

Pros	Cons
Adherence to DRY: Logic for Zapping a Tile is written once and reused by every emitter and resonator.	Static methods make unit testing harder (cannot mock via dependency injection).
High functional cohesion: ChargeUtils has single responsibility, by executing the Standard Impact of a surge.	Static coupling means all emitters are tightly bound to this implementation.
Adherence to SRP: Resonators like PoweredFloor focus on propagation (where the energy goes), while ChargeUtils focuses on impact (what the energy does).	While zapTile() combining damage, statuses, and inventory induction violates KISS.

Justification: Design 2 was chosen specifically to eliminate the Shotgun Surgery code smell present in Design 1. A stateless utility class, ChargeUtils, was chosen to centralize high-voltage operations. This eliminates the Shotgun Surgery code smell where changing a conceptual value, such as ShockedStatus duration would otherwise require edits across multiple emitter classes. While ChargeUtils.zapTile() handles multiple sub-responsibilities (damage, statuses, and inventory induction), this is a calculated cohesion trade-off. Consolidating the Standard Impact sequence ensures that if the interaction chain needs modification, it is changed in exactly one location, protecting the system's maintainability. Although centralising damage, statuses, and inventory induction into zapTile() violates KISS by giving one method multiple concerns, this is preferable to the alternative where every emitter must invoke all three operations in the correct sequence, spreading Connascence of Execution across every call site.

D. Material Identity via Capabilities (MaterialCapability Enum)

The design replaces brittle instanceof checks with a capability-tag system.

Design 1: Type Checking (instanceof / Downcasting)

Pros	Cons
Explicit: Directly shows which classes are affected.	Violation of OCP: Adding a new magnetic item would require adding a new instanceof check in the Wallet class.
No additional capability management overhead.	High coupling to concrete class names: brittle if class names change.
	Overuse of instanceof and downcasting can be considered a code smell, as it often indicates a violation of the Open-Closed

	Principle.
--	------------

Design 2: Capability-Tag System (MaterialCapability Enum)

Pros	Cons
Decoupling: Wallet doesn't need to know what a CRTMonitor is. It only needs to know that the item is MAGNETIC.	Capability assignment is implicit: Which classes hold which capabilities must be verified by checking constructors
Adheres to ISP: Classes only enable capabilities they actually need.	Capability strings are runtime-checked, losing compile-time safety.
Adheres to OCP: New magnetic items automatically work with Wallet by extending MagneticItem.	
Eliminates code smell of using instanceof.	

Justification: Design 2 was chosen to eliminate the instanceof code smell and strictly follow ISP and OCP. The design replaces brittle instanceof checks with a capability-tag system using the MaterialCapability enum. The MagneticItem abstract class centralizes capability registration, ensuring subclasses like CRTMonitor or Lantern automatically participate in magnetic harvesting (DRY). This allows the Wallet to interact with any MAGNETIC item without knowing its concrete class, fulfilling OCP. A future SteelBeam item would require only a new class declaration extending MagneticItem, requiring zero modifications to the Wallet or ChargeUtils.

E. Multi-Stage Terrain Morphing and Lifecycle Management

Objects like Puddle, ElectrifiedPuddle, and IonizedBarrier implement complex state changes with automatic reversion.

Design 1: Simple Terrain Swapping

Pros	Cons
KISS: Puddle simply calls <code>location.setGround(new ElectrifiedPuddle())</code> .	Unrealistic: If an actor occupies the puddle when struck, they feel no immediate effect. Damage applies to the next turn only.
Minimal complexity	No mechanism for lifespan refreshment, terrain restoration, or barrier reinforcement

Design 2: Multi-Stage Interaction

Pros	Cons
Adheres to SRP: <code>Puddle.reactToCharge()</code> handles transition logic in correct order: (Recursion Guard → Log Event → Zap Occupants → Morph Map).	Requires careful visited set management to ensure initial zap isn't blocked by morphing.
Maintainability: IonizedBarrier stores	Increased complexity: Three separate

previousGround, restoring original ground type (even if it was PoweredFloor) rather than default Floor.	lifecycle phases in ElectrifiedPuddle.tick().
Adheres to LSP: IonizedBarrier, ElectrifiedPuddle, and PoweredFloor can each stand in for any Ground reference without callers needing to know which specific subtype is present	
ElectrifiedPuddle implements ChargeReactive, allowing lifespan stacking rather than simple replacement, demonstrating realistic physics where repeated strikes intensify rather than reset the hazard	

Justification: Design 2 was chosen to demonstrate sophisticated, physically realistic environmental interaction and to satisfy LSP rigorously. The system demonstrates sophisticated environmental interaction and satisfies LSP through state-aware Ground objects. IonizedBarrier stores previousGround and restores it upon expiration, preserving map topology even if the original ground was a PoweredFloor. The added complexity of three lifecycle phases in ElectrifiedPuddle.tick() is justified because collapsing them would sacrifice lifespan stacking and immediate contact damage, reducing the system to a simple timer with no physical realism.

F. Status Effect Architecture (ShockedStatus / ParalyzedStatus)

The design encapsulates electrical side effects within dedicated status classes rather than scattering logic across actor turn methods, following the Single Responsibility Principle.

Design 1: Inline Status Logic (handle status effects directly in playTurn or tick)

Pros	Cons
Simple and no extra classes needed	Violates SRP: playTurn() would handle both decision-making AND electrical side effects
Easy to trace in a single method	Violates DRY: same arc to neighbours logic would be duplicated in ElectrifiedPuddle.tick(), StaticStalker.processStaticAura(), and any future hazard

Design 2: Encapsulated Status Classes with Cross-Component Behaviour

Pros	Cons
SRP: ShockedStatus handles its own damage, capability management, and arcing independently of the actor that holds it	ShockedStatus acts as both a Status and an implicit ChargeSource, giving it two loosely-related responsibilities
DRY: the arc to neighbours pulse in	Connascence of Execution: ShockedStatus

ShockedStatus.tickStatus() is written once and applied regardless of what caused the shocked state	must call ChargeUtils.triggerGroundReaction and ChargeUtils.zapTile in the correct order or risk double-damaging actors
OCP: new status effects (like in the future a MagneticSurgeStatus that spreads surge) can follow the same pattern without modifying any existing actor or ground class	
Without encapsulation, every hazard spawning a shocked actor would need to manually call enableAbility(CONDUCTIVE), apply damage, then trigger arcing which has three operations that must occur in sequence at every call site.	

Justification: Design 2 was chosen. ShockedStatus and ParalyzedStatus encapsulate electrical side effects, following SRP. ShockedStatus implements Status to inherit health attrition logic (DRY) while adding an AoE pulse. This layered inheritance satisfies LSP, as ShockedStatus stands in for a standard status while adding transparent arcing behavior. ParalyzedStatus simultaneously manages turn-skipping and the REFLECTIVE capability within a single lifecycle. Encapsulating these related behaviors prevents Connascence of Execution, as callers do not need to manually add multiple statuses in a specific order.

G: Magnetic Pulse Anomaly (DormantStaticCreature → StaticStalker Evolution)

Design 1: Immediate Evolution

Pros	Cons
Simplicity (KISS): Single method call transforms the creature. No state variables, no secondary methods. A developer understands it in 5 lines.	Loss of Player Anticipation: No warning before transformation. Players cannot react or prepare.
Low Memory Footprint: No additional instance variables. Scales efficiently for maps with many creatures.	Missed Gameplay Depth: Magnetic pulse mechanic (item repulsion) is absent. Removes risk-reward harvesting strategy.
Deterministic Behavior: Evolution occurs identically every time. Testing requires only one scenario.	Unrealistic Physics: A small puddle charge triggers the same evolution as a massive Tesla Coil. No energy scaling.
No Collateral Effects: Items remain undisturbed. Simplifies reasoning about side effects.	No Environmental Storytelling: No visual story of something powerful happened here.
Faster Execution: O(1) time complexity. No tile scanning or distance calculations.	Violates "Show, Don't Tell": Mechanics should be visually evident. Arbitrary transformation feels unearned.
SRP: Evolution is the class's single responsibility. No other concerns mixed in.	Violates OCP: Cannot add pulse effects or different evolution thresholds without

	modifying the class.
ISP: Clients depend only on the reactToCharge() method. No unused interface methods forced.	LSP Risk: Subclass cannot easily override or extend evolution behavior without breaking parent expectations.
Low Connascence: Only one place to change if evolution logic needs modification.	DIP Violation: Directly instantiates StaticStalker – coupled to concrete class, not abstraction.

Design 2: Delayed Evolution with Magnetic Pulse

Pros	Cons
Player Anticipation: 3-charge requirement creates predictable "ticking clock." Players decide: destroy, avoid, or deliberately trigger for rewards.	Increased Complexity (KISS Violation): 3 new methods, 1 instance variable, threshold constant. Class grows from ~50 to ~150 lines.
Emergent Reward System: Magnetic pulse repels MAGNETIC items toward the player. Risk-reward loop encourages proactive engagement.	Performance Overhead: Scans up to 25 tiles per evolution. Each tile checked for items. Potential lag with multiple simultaneous evolutions.
Physical Realism: Three charges before evolution models electrostatic buildup. Intuitive scaling: more energy = bigger reaction.	Boundary Checking Complexity: Must verify target tile in-bounds, passable, and not occupied. Multiple branching conditions.
Consistency with REQ3: Diamond AoE matches TeslaCoil's pattern. Unified "electrical language" across requirements.	
SRP: Responsibilities split across accumulateCharge(), executePulse(), repelItem(), evolve() – each has a single purpose.	
OCP: New pulse behaviors (e.g., stun instead of repel) can be added by overriding executePulse() without modifying core evolution.	
LSP: Subclasses like SuperchargedStalker can override executePulse() for stronger effects while preserving evolution contracts.	

Justification: Design 2 was chosen. Immediate evolution on first charge removes all player agency, a stray lightning bolt instantly creates a StaticStalker with no warning. The staged accumulation model instead creates anticipation: the creature visibly builds potential energy across multiple charges before erupting, giving players time to react. At anomalyRadius == 3, executeMagneticPulse() scans a diamond-shaped AoE using Manhattan distance ($\text{Math.abs(dx)} + \text{Math.abs(dy)} \leq \text{radius}$), matching TeslaCoil's geometric pattern for visual consistency. Crucially, the pulse repels MAGNETIC items — CRTMonitor, FloppyDisk,

Lantern, toward the player, turning a dangerous metamorphosis event into a dual-purpose mechanic: a threat and a scrap collection reward simultaneously. The MAGNETIC capability filter maintains OCP. Future magnetic items participate automatically without modifying `executeMagneticPulse()`.

H: ChargeContext and GalvanicCharge - Propagation Memory Architecture

Design 1: Depth-Limit Counter

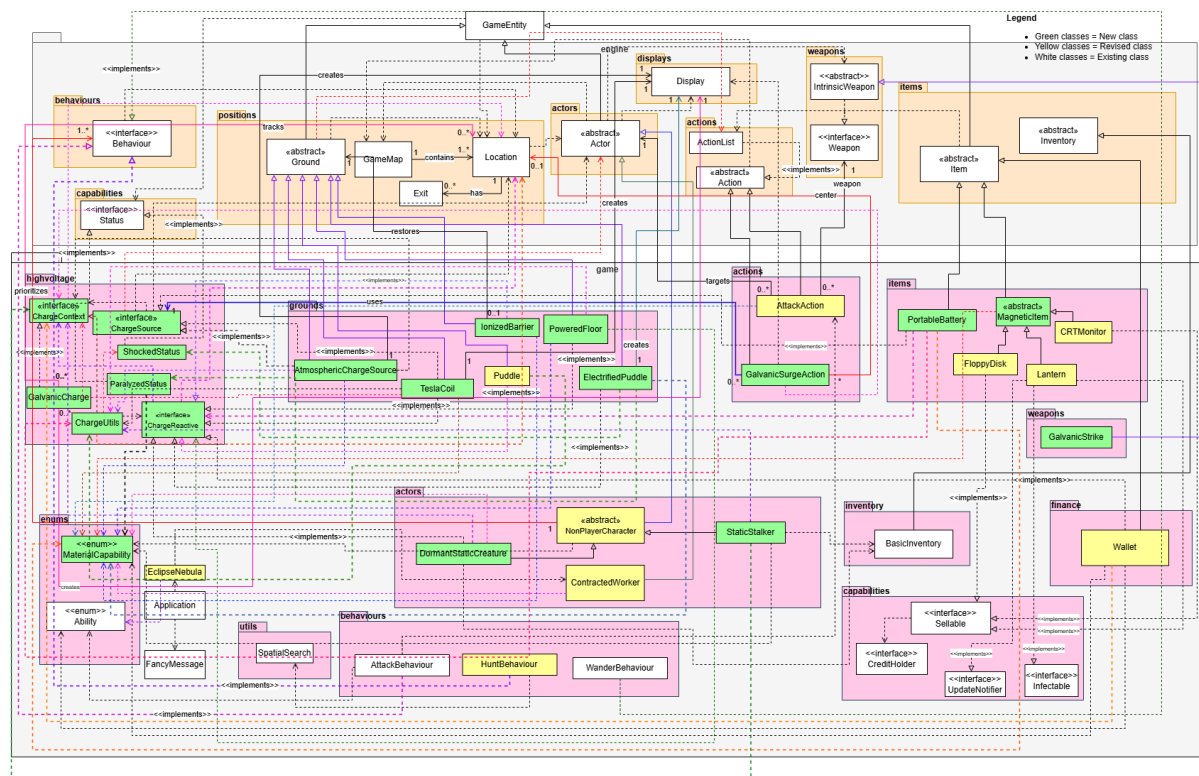
Pros	Cons
Simplicity (KISS): Single integer counter decrements with each propagation hop. No HashSet management, no memory overhead. A developer implements it in 3 lines.	Does Not Prevent Back-Arcing: Energy can flow $A \rightarrow B \rightarrow A$, decrementing counter twice but damaging both tiles twice. Creates duplicate damage and unrealistic "oscillating" current.
Low Memory Footprint: One integer (4 bytes) per surge. Scales perfectly for any map size. No object allocations during propagation.	Cannot Distinguish Revisits from Legitimate New Tiles: A tile might be legitimately reachable via two different paths (e.g., around a corner). Counter cannot tell if this is a new path or a loop returning.
Connascence of Value: Only one value (depth) must be agreed upon across all emitters and resonators.	DRY Violation: Every emitter and resonator must manually decrement and check the counter. Logic duplicated across TeslaCoil, PoweredFloor, Puddle, and ElectrifiedPuddle.

Design 2: Visited Set (HashSet<Location> in GalvanicCharge)

Pros	Cons
Adheres to LSP: Any ChargeContext implementation works without resonators knowing how memory is stored. A future OptimizedCharge could use bit arrays or Bloom filters without changing ChargeReactive implementers.	Memory Overhead per Surge: Each lightning strike or coil pulse creates a new HashSet. For a map with 100 tiles, a worst-case surge storing 100 Locations is a lot. Acceptable for modern hardware.
DRY Compliance: <code>visit()</code> logic centralized in <code>GalvanicCharge.visit()</code> and <code>ChargeUtils.zapTile()</code> . Emitters don't implement their own duplicate detection.	Connascence of Algorithm: All emitters and resonators must agree on <code>visit()</code> call timing. If <code>PoweredFloor</code> calls <code>visit()</code> but <code>TeslaCoil</code> doesn't, safety guarantees break.
Guaranteed No Infinite Recursion: HashSet prevents any tile from being processed twice in the same surge. $A \rightarrow B \rightarrow A$ pattern: second A is rejected by <code>visited.contains(A)</code> . No <code>StackOverflowError</code>	

Justification: Design 2 selected. Despite higher complexity and memory overhead, the visited set provides guaranteed safety (no infinite recursion, no duplicate damage) that a depth-limit cannot offer. Physical realism and testability are superior. The high memory cost

per surge is negligible for modern hardware, and the centralized ChargeUtils.visit() pattern mitigates connascence concerns by providing a single source of truth for the safety check.



The diagram above shows the final design of Requirement 3, which utilizes each Design 2 of parts A, B, C, D, and E stated above to implement a complete high-voltage galvanic ecosystem with recursive propagation, centralized impact coordination, capability-based identification, multi-stage terrain morphing, and single-threaded action execution.

The new ChargeSource and ChargeReactive interfaces act as a level of abstraction between emitters (AtmosphericChargeSource, TeslaCoil, PortableBattery) and resonators (Puddle, ElectrifiedPuddle, PoweredFloor, IonizedBarrier, DormantStaticCreature, Wallet), removing dependencies between concrete classes without changing the flow of control by diverting dependencies to depend on abstractions (Dependency Inversion Principle). This increases extensibility. If we were to add new electrical sources in future development, no modifications need to be made to existing resonators (Open-Closed Principle).

ChargeUtils segregates responsibility. Emitters focus on where energy goes (propagation patterns: diamond AoE, map-wide lightning, or single-use battery blasts), while ChargeUtils focuses on what energy does (damage, status effects, metamorphosis, inventory induction). Using this implementation, each class is responsible for its own purpose only (Single Responsibility Principle). The MaterialCapability enum eliminates instance of code smells, instead of checking if (item instanceof CRTMonitor), the system checks if (item.hasAbility(MaterialCapability.MAGNETIC)). This allows any new scrap item extending MagneticItem to automatically participate in magnetic harvesting without modifying Wallet (Open-Closed Principle).

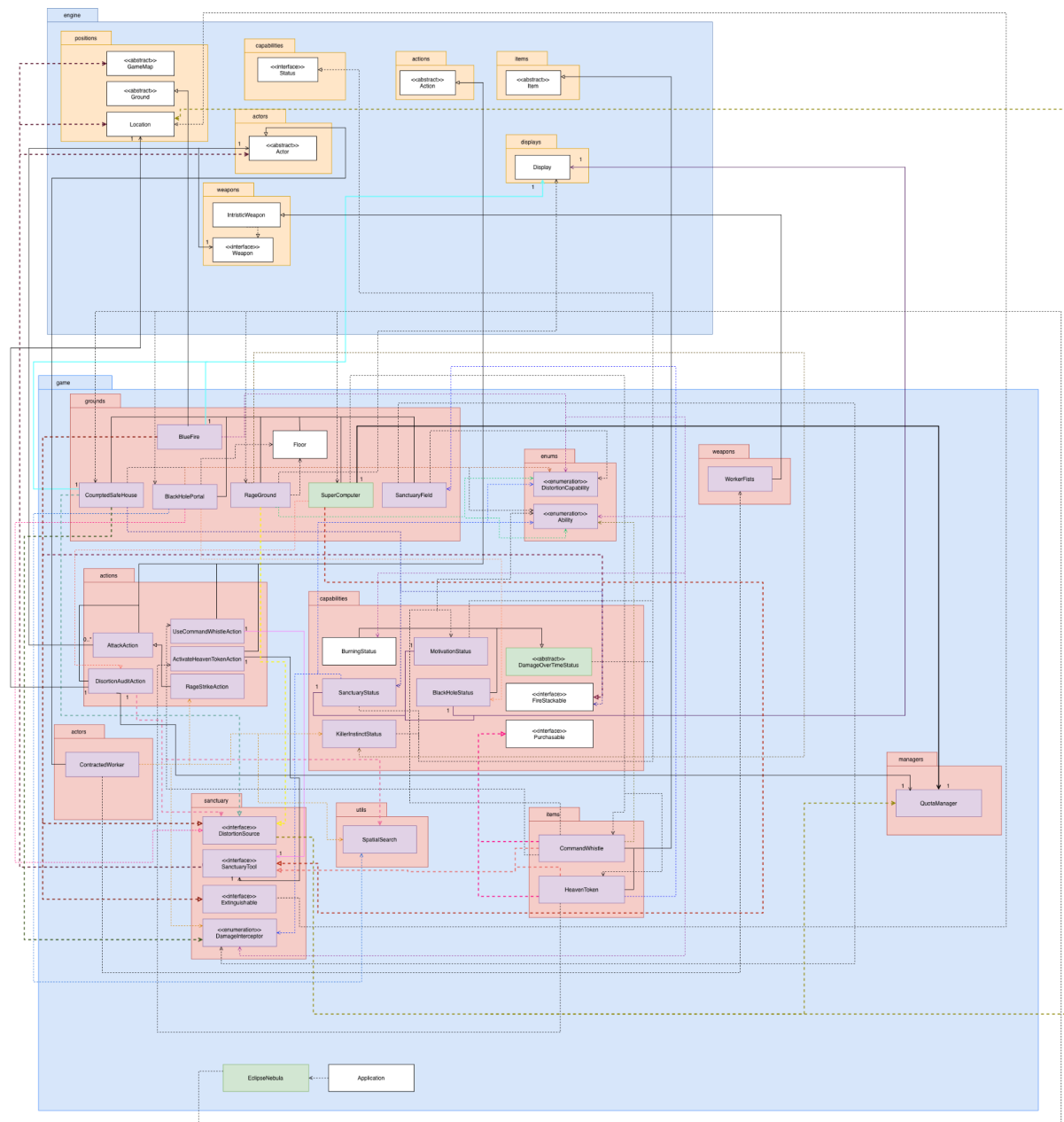
The GalvanicCharge class maintains a HashSet<Location> visited set, providing memory of which tiles have been processed during a surge event. This prevents infinite recursion between adjacent PoweredFloor tiles and ensures each entity is damaged only once per lightning strike (robustness). Any ChargeContext implementation works without resonators knowing how memory is stored. This represents a significant architectural evolution from A1/A2, where interactions were often hard-coded within tick() methods using instanceof. In A3, the High-Voltage system uses Strategy and State patterns (via ChargeReactive and Status effects like ParalyzedStatus and ShockedStatus) to ensure the environment is alive and interconnected, sharing a single source of truth for electrical logic across REQ3.

Changes from Assignment 2

Class	Change	Justification
AttackAction	Added reflective feedback when target has REFLECTIVE capability	Enables electrical arc-back mechanic when paralyzed actors are attacked
ContractedWorker	Added isParalyzed() check in turn processing	Prevents worker from acting when stunned by high-voltage
NonPlayerCharacter	Added paralysis check in playTurn()	Ensures all NPCs respect the PARALYZED capability
Wallet	Added MAGNETIC capability and ChargeReactive interface	Allows wallet to act as electromagnet when powered
Puddle	Added ChargeReactive interface	Enables terrain morphing into ElectrifiedPuddle
CRTMonitor, FloppyDisk, Lantern	Changed to extend MagneticItem	Centralizes magnetic behavior and locking logic

The system represents an architectural evolution from hard-coded interactions to a Strategy-based ecosystem. Extracting magnetic behavior into MagneticItem resolved a code smell where enableAbility was duplicated across constructors. By using asCapability(ChargeReactive.class) and visited sets, Requirement 3 implements a robust, interconnected environment that shares a single source of truth for galvanic logic.

UML DIAGRAM - REQ4



DESIGN RATIONALE - REQ4

A. Design of the DistortionSource Interface

Design 1: Direct concrete class coupling in `DistortionAuditAction` using `instanceof`

Pros	Cons
------	------

Simpler to implement as no interface or extra abstraction needed.	DIP violated: DistortionAuditAction imports BlackHolePortal, CorruptedSafeHouse, and RageGround directly. Adding a new anomaly requires modifying the action class, an OCP violation.
All audit logic is visible in a single class, making tracing easy.	SRP violated: the action class takes on responsibility for the internal logic of every anomaly type.

Design 2 (Chosen): DistortionSource interface with stabilise(), audit(); getGroundAs() used in DistortionAuditAction

The DistortionSource interface defines two methods: `stabilise()` for SuperComputer triggered cleanup, and `audit()` for the corporate Distortion Audit Protocol. `DistortionAuditAction` uses `SpatialSearch.getLocationsWithinDistance()` for a radius 3 scan and calls `loc.getGroundAs(DistortionSource.class)` on each tile. Each anomaly implements `audit()` to handle its own credit contribution, final unstable effect, and cooldown state.

Pros	Cons
DIP satisfied: <code>DistortionAuditAction</code> calls <code>getGroundAs(DistortionSource.class)</code> as it never imports or references any concrete anomaly class.	Requires all three anomaly classes to implement a two method interface.
OCP satisfied: adding a new anomaly only requires implementing <code>DistortionSource</code> ; the audit action changes zero lines.	Developers must understand the interface contract to add new anomalies; missing an override compiles but does nothing.
SRP: each anomaly class encapsulates its own <code>audit()</code> logic. The action only performs the scan and dispatch.	
Consistent with the engine's capability pattern as <code>getGroundAs()</code> , does not use <code>instanceOf</code>	
The two method contract cleanly separates stabilisation, and audit into distinct interaction modes.	

Final Design Justification

Design 2 was chosen. The `DistortionSource` interface creates a single stable abstraction that `DistortionAuditAction` depends on exclusively. In `DistortionAuditAction.execute()`, `SpatialSearch.getLocationsWithinDistance()` collects tiles in a radius-3 area, and `loc.getGroundAs(DistortionSource.class)` queries each tile without any instanceof or concrete class reference (DIP). Each anomaly's `audit()` method handles its own credit contribution, final effect, and cooldown state (SRP, LSP).

B. Implementation of the CorruptedSafeHouse Presence Aware Lifecycle

Design 1: Poll presence in tick() and manage both status and BlueFire spawning directly inline

Pros	Cons
Simple: all logic lives in a single tick() method and is easy to trace.	SRP violated: tick() takes on simultaneous responsibility for status management, BlueFire spawning, audit cooldown, and fire cleanup.
	The interaction between SanctuaryStatus protection turns and BlueFire lifespan becomes tightly coupled as changing one requires careful re reading of the other.
	Testing individual concerns (e.g., just the heal chance) requires simulating the entire tick() path.

Design 2 (Chosen): tick() delegates to spawnBlueFire(); SanctuaryStatus owns its own lifecycle; FireStackable interface handles fire refresh

tick() orchestrates presence detection, SanctuaryStatus granting, and BlueFire spawning by delegating to the private spawnBlueFire() method. spawnBlueFire() uses getGroundAs(FireStackable.class) to detect existing fire and calls fire.addStack() to intensify it, or places a new BlueFire if the tile is empty and safe. SanctuaryStatus manages its own protection turn counter and expiry logic independently. The auditCooldown field pauses the full lifecycle during the weakened post audit window.

Pros	Cons
SRP: tick() orchestrates; spawnBlueFire() handles fire spawning detail. Each method has one job.	Two classes (CorruptedSafeHouse and SanctuaryStatus) share responsibility for the protection lifecycle, requiring a developer to read both to understand the full behaviour.
FireStackable.addStack() allows CorruptedSafeHouse to intensify existing BlueFire rather than replacing it. No instanceof required, uses getGroundAs(FireStackable.class) idiomatically (DIP).	The spawnBlueFire guard conditions (ACTIVE_HAZARD check, CORRUPTED check, canActorEnter) must be correctly ordered to avoid logical errors.

SanctuaryStatus owns its own protection turn counter and expiry logic, so CorruptedSafeHouse does not need to track it.	
ACTIVE_HAZARD capability on BlueFire prevents the safe house from overwriting existing hazards, using the capability system rather than type checks.	
The auditCooldown integer is a single self-decrementing counter that pauses the full lifecycle as this is easy to reason about and test independently.	

Final Design Justification

Design 2 was chosen. The tick() method in CorruptedSafeHouse follows a clean orchestration pattern: it checks for the worker, grants SanctuaryStatus if absent, and delegates fire spawning to spawnBlueFire(). The FireStackable interface (implemented by BlueFire) allows the safe house to refresh fire intensity via fire.addStack() when getGroundAs(FireStackable.class) returns a result, avoiding any instanceof check and remaining fully extensible (OCP). SanctuaryStatus manages its own 5-turn protection window and self-destructs when the worker leaves the CORRUPTED tile, ensuring a full reset on re entry. The auditCooldown field is the only shared state, and its semantics are explicit: during cooldown the safe house suppresses all new effects while existing statuses continue to tick naturally.

C. Implementation of BlueFire (Terrain Restoration and Fire Intensification)

Design 1: BlueFire stores only the original ground's display character and restores a Floor on extinguish

Pros	Cons
Simple as no extra reference storage beyond a character.	Information loss: the original ground type (Dirt, Floor, or any variant) is discarded. A Floor is restored regardless, which is incorrect if BlueFire replaced a non-floor tile.
	Cannot support arbitrary original ground types without expanding the stored state.

Design 2 (Chosen): BlueFire stores a Ground originalGround reference. FireStackable interface for intensification via addStack()

BlueFire accepts a Ground originalGround parameter in its constructor. `extinguish(location)` calls `location.setGround(originalGround)`, faithfully restoring whatever ground was there before. The `ACTIVE_HAZARD` capability flag on BlueFire prevents CorruptedSafeHouse from double-spawning fire by checking `hasAbility(ACTIVE_HAZARD)` before placing new fire. `isSafeHouseOccupied()` checks adjacent tiles for a `CORRUPTED` ground with a `WORKER` actor using capability checks exclusively.

Pros	Cons
originalGround stores the location's previous ground state. When <code>extinguish()</code> is called, it restores that state using <code>location.setGround(originalGround)</code> , allowing any ground type such as Dirt, Floor, or future ground types to be reinstated without requiring changes to the restoration logic, thereby supporting the Open/Closed Principle (OCP).	Storing a Ground reference means the originalGround object must not be mutated after BlueFire is placed; if tile state changes while BlueFire is active, restore would roll back to a stale state.
<code>FireStackable.addStack()</code> in <code>CorruptedSafeHouse</code> is called via <code>getGroundAs(FireStackable.class)</code> has no instanceof, pure DIP.	The <code>MAX_LIFESPAN</code> limit in <code>addStack()</code> prevents the effect's duration from increasing beyond a predefined maximum value. This behavior is intentional to avoid unbounded stacking, although it may be unexpected for developers who assume the duration can continue increasing indefinitely.
<code>ACTIVE_HAZARD</code> capability flag prevents double spawning using the capability system rather than type inspection.	
<code>isSafeHouseOccupied()</code> checks adjacent tiles for <code>CORRUPTED</code> ground with a <code>WORKER</code> actor using pure capability checks, no class names (DIP).	
<code>Extinguishable</code> interface provides a clean contract for any system that needs to remove a ground hazard programmatically (e.g., the audit protocol).	

Final Design Justification

Design 2 was chosen. The originalGround constructor parameter stores the tile's previous ground state when it is replaced by BlueFire. When `extinguish()` is called, the original state is restored through `location.setGround(originalGround)`. Because the exact ground object is

reinstated rather than recreated, the solution works correctly for any current or future ground type, supporting the Open/Closed Principle (OCP). The FireStackable interface enables CorruptedSafeHouse.spawnBlueFire() to invoke fire.addStack() on an existing BlueFire tile retrieved via getGroundAs(FireStackable.class), refreshing its lifespan to MAX_LIFESPAN instead of creating a duplicate fire. Additionally, the ACTIVE_HAZARD capability serves as an efficient safeguard against overwriting existing hazards without requiring any instanceof checks.

D. Implementation of RageGround (Action Mutation and Terrain Relocation)

Design 1: Inject RageStrikeAction directly in RageGround.tick() via the actor's action list

Pros	Cons
Simple as tick() handles both status grant and action injection in one place.	Ground.tick() receives only a Location, not an ActionList. The engine does not pass the action list to ground tick methods, making direct injection structurally impossible without engine modification.
	Coupling the ground to action menu management violates SRP.

Design 2 (Chosen): tick() grants KillerInstinctStatus; ContractedWorker.playTurn() injects RageStrikeActions when the status is present

RageGround.tick() detects a worker on the tile, adds a KillerInstinctStatus for KILLER_INSTINCT_SPAN turns, and immediately calls relocate() which replaces the current tile with Floor and places a new RageGround at a random valid location. ContractedWorker.playTurn() checks hasStatus(KillerInstinctStatus.class) using class comparison and calls injectRageStrikeActions(), which uses SpatialSearch.getActorsWithinDistance() to find targets within RAGE_STRIKE_RADIUS and adds a RageStrikeAction for each. stabilise() removes KillerInstinctStatus from the worker on the tile via a statuses() list scan with class comparison, not instanceof.

Pros	Cons
Architecturally correct: tick() has access to Location and its actor; playTurn() has access to the ActionList. Each method does only what it can do (SRP).	The injection logic is split across two classes (RageGround and ContractedWorker). A developer must read both to understand the full effect chain.

KillerInstinctStatus.isStatusActive() controls the injection window precisely as RageStrikeActions appear for exactly as many turns as the status lasts.	The RAGE_STRIKE_RADIUS constant is defined in ContractedWorker rather than RageGround, which slightly obscures the relationship between the ground and its scan range.
injectRageStrikeActions() uses SpatialSearch.getActorsWithinDistance() with a target != this guard uses no instanceof, no capability check needed (DIP).	
hasStatus(KillerInstinctStatus.class) uses class comparison, not instanceof, consistent with the project constraint.	
stabilise() removes KillerInstinctStatus via statuses() scan with class comparison, cleanly cleaning up on SuperComputer audit.	

Final Design Justification

Design 2 was chosen. Ground.tick() is the correct site to detect worker presence and grant status; ContractedWorker.playTurn() is the correct site to inspect status and modify the action list. This division respects the engine's architecture (SRP). KillerInstinctStatus acts as the communication channel between the two classes: it carries the duration, ticks down automatically, and its presence or absence gates the injection. The terrain relocation in relocate() uses a do-while loop that checks canActorEnter(null), !containsAnActor(), and !hasAbility(CORRUPTED). All capability based guards that avoid instanceof and correctly exclude existing distortion zones from being overwritten.

E. Implementation of SanctuaryStatus (Presence Aware Protection Lifecycle)

Design 1: CorruptedSafeHouse.tick() manually removes the status when no worker is present

Pros	Cons
Centralises all safe house logic in one class.	SRP violated: the ground class takes on responsibility for managing status expiry in addition to spawning effects.
	Race condition risk: if SanctuaryStatus.tickStatus() fires before the ground's tick(), the protection is active for one extra turn after departure.
	SanctuaryStatus functions solely as a passive data holder and does not manage

	its own state or behavior, resulting in a less self-contained design where the data and the logic that operates on it are separated.
--	--

Design 2 (Chosen): SanctuaryStatus.tickStatus() checks ground capability and actor ability; self-destructs on departure

SanctuaryStatus.tickStatus() first checks that the actor has WORKER ability and that the current tile has the CORRUPTED capability. If either check fails, the status sets remainingTurns to 0 and calls actor.disableAbility(DamageInterceptor.PROTECTED), self-destructing immediately. While active, the protection turn counter decrements each tick; once it reaches zero, isProtectionActive() returns false, signalling CorruptedSafeHouse to stop spawning BlueFire, while the status itself continues ticking for healing. ContractedWorker.hurt() reads the PROTECTED flag and blocks all damage when set.

Pros	Cons
SRP: SanctuaryStatus owns its full lifecycle. It decides when it is active, when protection is enabled, and when it should expire.	The PROTECTED ability is updated every tick according to the current status instead of being assigned once. If multiple status effects influence PROTECTED, this repeated update pattern could cause conflicting outcomes.
Departure detection via location.getGround().hasAbility(DistortionCapability.CORRUPTED) is a pure capability check and no instanceof, no concrete class reference used (DIP).	The status has two separate counters: isActive() for overall liveness and isProtectionActive() for the protection window. This dual counter design requires clear documentation to avoid confusion.
isProtectionActive() gates BlueFire spawning in CorruptedSafeHouse; once protection turns run out, fire stops spawning automatically without any change to the ground's logic.	
Self destruction on a non CORRUPTED tile ensures the next entry to the safe house gets a fresh status with full protection turns.	
actor.hasAbility(Ability.WORKER) check in tickStatus() prevents the status from being applied to non-workers.	

Final Design Justification

Design 2 was chosen. `SanctuaryStatus.tickStatus()` is the correct site for the departure check because `Status.tickStatus()` is called by the engine every turn for every entity, giving the status full autonomy over its lifecycle without any ground-side polling. The departure check `location.getGround().hasAbility(DistortionCapability.CORRUPTED)` is a single capability query that works regardless of which `CORRUPTED` ground type the actor is standing on (DIP, OCP). The dual counter design maps precisely to the two separate requirement phases: the protection active phase and the healing-only phase.

`ContractedWorker.hurt()` reads `DamageInterceptor.PROTECTED` and blocks all damage when the flag is set, keeping the protection logic entirely out of the status class itself (SRP).

F. Implementation of the CommandWhistle (Remote Knockback Pulse)

Design 1: Execute full knockback physics in `UseCommandWhistleAction.execute()`

Pros	Cons
All whistle logic is in one place as it is easy to trace.	SRP violated: the action class takes on target scanning, status application, knockback physics, and collision damage simultaneously.
	The knockback fires on the same turn the whistle is used. The intended one-turn deferred firing requires a status-based mechanism, which this design cannot support.
	Testing the knockback physics requires constructing a full action and map which results in coupling test complexity to the action.

Design 2 (Chosen): `activateSanctuaryEffect()` scans and applies `MotivationStatus`; `MotivationStatus.tickStatus()` owns all knockback physics

`CommandWhistle.activateSanctuaryEffect()` calls `findNearestFriendly()` using `getNearbyLocations(SEARCH_RADIUS)` and `hasAbility(Ability.WORKER)` to locate the closest allied worker. If none is found within range, the whistle user is the target. `MotivationStatus` is applied to the target with the whistle user's location as the pulse origin. On the first tick, `MotivationStatus` iterates exits from that origin, pushes non worker actors 2 tiles via `findKnockbackDest()`, and deals `IMPACT_DAMAGE` on a blocked path. The `hasPulsed` flag ensures the pulse fires exactly once.

Pros	Cons
------	------

SRP: CommandWhistle.activateSanctuaryEffect() owns only 'find target + apply status'. MotivationStatus.tickStatus() owns all knockback physics.	The pulse fires on the status's next tick, meaning there is a one-turn delay between using the whistle and the knockback. This is by design but may confuse players.
MotivationStatus stores the origin at construction time so the pulse always radiates from the whistle user's tile regardless of where the motivated actor stands.	MotivationStatus is a one shot status; if the actor is removed from the map before the tick fires, the pulse is lost. This edge case is acceptable for the current game scope.
hasPulsed flag ensures the status fires exactly once and then expires automatically (isStatusActive() returns false) with no cleanup needed.	
Ability.WORKER check on adjacent actors avoids instanceof (DIP). Collision detection navigates exits by name and no map size assumptions.	
The SanctuaryTool interface means UseCommandWhistleAction calls tool.activateSanctuaryEffect() and never imports CommandWhistle directly (DIP).	

Final Design Justification

Design 2 was chosen. Separating the scan from the physics is a direct application of SRP. The SanctuaryTool interface further decouples the action layer: UseCommandWhistleAction simply calls tool.activateSanctuaryEffect() and forwards the result without importing CommandWhistle directly (DIP). The origin parameter passed to MotivationStatus at construction captures the whistle user's position for the pulse centre regardless of where the motivated actor later stands. findNearestFriendly() uses getNearbyLocations(SEARCH_RADIUS) and checks hasAbility(Ability.WORKER) therefore no instanceof, extensible to any future worker type automatically (OCP).

G. Implementation of the HeavenToken

Design 1: ActivateHeavenTokenAction stores the SanctuaryField duration and replaces ground directly inline

Pros	Cons
Simple as the action class has the map and location, so replacing the ground inline is straightforward.	SRP violated: the action class manages both item consumption logic and terrain mutation logic.

	The previousGround reference must be captured before the replacement occurs. However, placing this responsibility in the action ties terrain restoration details to the action itself rather than encapsulating them within the HeavenToken
--	---

Design 2 (Chosen): HeavenToken.activateSanctuaryEffect() captures previousGround, creates SanctuaryField, and removes itself from inventory

HeavenToken.activateSanctuaryEffect() captures location.getGround() as previousGround, then replaces the tile with new SanctuaryField(FIELD_DURATION, previousGround) and removes itself from the actor's inventory. ActivateHeavenTokenAction calls only tool.activateSanctuaryEffect() and never imports HeavenToken. The AoE protection lifecycle as clearAndReapply(), turn-based expiry, and terrain restoration and is fully owned by SanctuaryField (see Section J).

Pros	Cons
SRP: the token owns its own activation and self removal. The action class only calls activateSanctuaryEffect() and forwards the result.	SanctuaryField.tick() uses a clear and reapply pattern (disable then re-enable PROTECTED) each turn to handle actors entering and leaving mid duration. This is correct but slightly more expensive than a single enable.
SanctuaryField stores previousGround in its constructor, using the same restoration approach as BlueFire, ensuring the original tile is restored correctly after 10 turns.	
SanctuaryField.tick() applies PROTECTED to the centre and adjacent tiles each turn, encapsulating the AoE logic entirely (SRP).	
The SanctuaryTool interface means ActivateHeavenTokenAction never imports HeavenToken directly (DIP), and the same action pattern works for any future SanctuaryTool item.	

Final Design Justification

Design 2 was chosen. The HeavenToken follows the same SanctuaryTool contract as CommandWhistle: the item owns the activation logic, the action class is kept thin. activateSanctuaryEffect() is the correct site to capture previousGround and place SanctuaryField, because the token holds the semantic meaning of "deploy a sanctuary here." The SanctuaryTool interface means ActivateHeavenTokenAction never imports

HeavenToken directly (DIP), and the same action pattern works for any future SanctuaryTool item (OCP). All AoE protection and terrain restoration behaviour is delegated to SanctuaryField, see Section J.

H. Implementation of BlackHoleStatus (Reusing DamageOverTimeStatus)

Design 1: Standalone BlackHoleStatus reimplementing damage and adding teleportation inline

Pros	Cons
Self contained as all logic in one class.	DRY violated: the 1 HP per-turn decrement, turn counter, death check, and display output are duplicated from DamageOverTimeStatus, which is already tested and used elsewhere.
	Any fix to the base damage logic must be applied in two places.

Design 2 (Chosen): BlackHoleStatus extends DamageOverTimeStatus; tickStatus() calls super then adds random map wide teleportation

BlackHoleStatus extends DamageOverTimeStatus, passing the warp duration to the parent constructor. tickStatus() calls super.tickStatus() to apply 1 HP of damage and decrement the turn counter, then calls entity.asCapability(Actor.class) to retrieve the actor and attempts teleportation via findSafeWarpLocation(). findSafeWarpLocation() loops up to MAX_WARP_ATTEMPTS times, checking canActorEnter(null) and !containsAnActor() on randomly selected tiles. BlackHolePortal.tick() guards the status add with !actor.hasStatus(BlackHoleStatus.class) to prevent stacking.

Pros	Cons
DRY: the damage tick, turn decrement, death check, and console output are all inherited from DamageOverTimeStatus with zero duplication.	Extends a concrete class rather than an interface, creating tighter coupling to DamageOverTimeStatus's implementation. If the parent class changes its tick() contract, the override must be updated.
The super.tickStatus() then additional logic override pattern clearly signals the intent: extend existing behaviour without replacing it.	The teleportation branch depends on the super call's death check having already fired; the ordering between damage and teleport must be carefully maintained.
entity.asCapability(Actor.class) avoids instanceof in the teleportation branch (DIP).	

findSafeWarpLocation() uses canActorEnter(null) and !containsAnActor() and is purely state based guards, no class checks.	
MAX_WARP_ATTEMPTS prevents infinite loops in dense maps by returning null as a safe fallback with no teleportation.	

Final Design Justification

Design 2 was chosen. Extending DamageOverTimeStatus is consistent with how InfectionStatus is implemented in the same codebase and is the correct DRY pattern for adding behaviour on top of an existing tick structure. super.tickStatus() handles all damage accounting and turn management; the override adds only the spatiotemporal warp. The asCapability(Actor.class) call in the override body means BlackHoleStatus never references ContractedWorker or any other concrete actor class (DIP). BlackHolePortal.tick() gates the status add with !actor.hasStatus(BlackHoleStatus.class), preventing stacking, and the auditCooldown field in BlackHolePortal pauses status application during the weakened post-audit window.

I. Implementation of the DistortionCapability Enum (DIP Facilitation)

Design 1: Add CORRUPTED, SANCTUARY, and ACTIVE_HAZARD to the existing Ability enum

Pros	Cons
No new enum file required.	Ability already carries actor-level capabilities (WORKER, IS_FLOOR, etc.). Adding ground-level distortion tags to the same enum reduces cohesion and makes the enum harder to understand.
	Unrelated concerns share a namespace as a future actor could accidentally be assigned CORRUPTED with no compile time warning.

Design 2 (Chosen): Dedicated DistortionCapability enum with CORRUPTED, SANCTUARY, and ACTIVE_HAZARD

DistortionCapability is a standalone enum with three values. CORRUPTED is enabled on BlackHolePortal, CorruptedSafeHouse, RageGround, and BlueFire to allow the SuperComputer to detect auditable anomalies via getGroundAs(DistortionSource.class). SANCTUARY is enabled on SanctuaryField to distinguish protective zones. ACTIVE_HAZARD is enabled on BlueFire so CorruptedSafeHouse.spawnBlueFire() can skip already-burning tiles without any type check.

Pros	Cons
Separation of concerns: distortion-related tags are namespaced independently of actor capabilities, making each enum's purpose clear.	Adds a new enum file. For three values, this may seem like unnecessary overhead compared to reusing the Ability enum.
All three values have precise, single-consumer semantics: CORRUPTED for audit scanning, SANCTUARY for zone identification, ACTIVE_HAZARD for fire-spawn guarding.	
hasAbility(DistortionCapability.CORRUPTED) in DistortionAuditAction and SanctuaryStatus replaces any instanceof check for anomaly grounds (DIP).	
Follows the established project pattern (Ability, MaterialCapability, FacilityCapability) as DistortionCapability is immediately understandable to any developer familiar with the codebase.	

Final Design Justification

Design 2 was chosen. The DistortionCapability enum is the linchpin that allows the entire REQ4 system to operate without instanceof. CORRUPTED enables the SuperComputer's Distortion Audit Protocol to locate anomalies via getGroundAs(DistortionSource.class); ACTIVE_HAZARD prevents BlueFire stacking; SANCTUARY identifies SanctuaryField tiles. Each value has a single, unambiguous consumer. Following the existing enum-as-capability pattern ensures consistency across the codebase and makes the system immediately understandable to any developer familiar with the engine's capability model (DIP, OCP).

J. Implementation of SanctuaryField (AoE Terrain Protection)

Design 1: Inline ground replacement in ActivateHeavenTokenAction.execute(), no dedicated class

Pros	Cons
Simple as the action already has the map and location, so replacing the ground and manually enabling PROTECTED on the occupant is straightforward.	SRP violated: the action class manages item consumption, ground replacement, actor protection, and turn based expiry simultaneously.

	No clean site for tick-based AoE re-application. The engine calls Ground.tick() each turn; putting re-application logic in the action means it only fires once, making the protection single turn only.
	previousGround restoration requires the action to store a reference it has no natural lifecycle for.

Design 2 (Chosen): Dedicated SanctuaryField class with clearAndReapply() and previousGround constructor parameter

SanctuaryField accepts duration and previousGround in its constructor. Each turn, tick() calls clearAndReapply() on the centre tile and all adjacent exits, which disables then enables DamageInterceptor.PROTECTED on any actor present. When remainingTurns reaches zero, clearProtectionAt() is called on the centre and all exits before location.setGround(previousGround) restores the original tile.

ActivateHeavenTokenAction.execute() only calls tool.activateSanctuaryEffect() and sets the immediate turn PROTECTED flag as it never manages expiry.

Pros	Cons
SRP: SanctuaryField owns the full AoE protection lifecycle. The action class is kept thin (calls one method, sets one flag).	clearAndReapply() disables then re-enables PROTECTED every tick, which is slightly more expensive than a one-time grant. This is necessary to handle actors who enter or leave mid-duration, but adds overhead
The clearAndReapply() pattern correctly handles actors entering and leaving the field mid-duration without any per actor state tracking.	
The previousGround parameter uses the same stat -preservation approach as BlueFire, establishing a consistent terrain replacement convention across REQ4.	
DistortionCapability.SANCTUARY on SanctuaryField allows other systems to identify protective zones via capability check with no instanceof.	

Final Design Justification

Design 2 was chosen. A dedicated SanctuaryField class is necessary because the AoE protection must re apply every turn as something only achievable via Ground.tick(). The clearAndReapply() loop iterates over location.getExits() to apply PROTECTED to the centre and all 8 adjacent actors each turn, encapsulating the AoE logic entirely (SRP). The previousGround constructor parameter follows the identical Memento pattern used by BlueFire, ensuring consistent terrain restoration semantics across the system: any ground replaced during REQ4 is faithfully reinstated by calling location.setGround(originalGround) on expiry, supporting any future ground type without modification (OCP).

K. Implementation of RageStrikeAction (Injected Combat Mutation)

Design 1: Modify AttackAction with a boolean flag to enable lifesteal mode

Pros	Cons
Reuses the existing attack class as no new class needed.	SRP violated: AttackAction acquires responsibility for both normal combat and lifesteal mutation, two behaviours with different triggers and semantics.
	OCP violated: modifying AttackAction to accommodate RageGround's mutation contaminates an unrelated class with REQ4 specific logic.
	The flag must be threaded through ContractedWorker to AttackAction, coupling the worker's state to the action's internal mode.

Design 2 (Chosen): Separate RageStrikeAction class injected into the worker's menu by injectRageStrikeActions()

RageStrikeAction wraps WorkerFists (the worker's intrinsic weapon) and, on execution, applies the attack and heals the attacker by the damage dealt.

ContractedWorker.injectRageStrikeActions() uses SpatialSearch.getActorsWithinDistance() with a target != this guard to inject one RageStrikeAction per in range actor into the ActionList when KillerInstinctStatus is present. The action appears in the menu for exactly as many turns as KillerInstinctStatus remains active and disappears automatically when the status expires.

Pros	Cons
SRP: RageStrikeAction owns only lifesteal combat logic. AttackAction is unchanged.	The injection logic is split between RageGround.tick() (status grant), ContractedWorker.playTurn() (injection trigger), and RageStrikeAction.execute()(combat + heal), requiring a developer to trace across three classes.

OCP: AttackAction requires zero modification for REQ4 combat mutation.	
The action is scoped precisely to KillerInstinctStatus's lifespan and no cleanup needed, the action simply stops being injected once the status expires.	
getActorsWithinDistance() uses hasAbility() checks and no instanceof, consistent with the DIP constraint.	

Final Design Justification

Design 2 was chosen. Separating RageStrikeAction from AttackAction is the correct application of SRP and OCP. AttackAction is a shared class used by multiple actors; coupling it to RageGround's mutation would require every future REQ4 variant to modify a core class. RageStrikeAction is self contained: it takes a target, a direction, and the worker's intrinsic weapon as the same constructor pattern as AttackAction and adds only the lifesteal on top. injectRageStrikeActions() uses SpatialSearch.getActorsWithinDistance() with a radius-3 scan and a target != this guard, both of which are pure capability and identity checks with no instanceof (DIP).

L. Implementation of the SanctuaryTool Interface

Design 1: Each tool item implements its activation logic directly in allowableActions()

Pros	Cons
Simple as no interface needed; each item's action is self-contained.	DIP violated: UseCommandWhistleAction and ActivateHeavenTokenAction must import CommandWhistle and HeavenToken directly, coupling action classes to concrete item types.
	OCP violated: adding a new tool item requires creating a new action class that imports the concrete item, and SuperComputer.allowableActions() must be modified to list the new purchase option.
	Duplicates the "thin action delegates to item" pattern across every tool.

Design 2 (Chosen): SanctuaryTool interface with activateSanctuaryEffect(); action classes depend only on the interface

SanctuaryTool defines a single method: `activateSanctuaryEffect(Actor, GameMap, Location)`. `CommandWhistle`, `HeavenToken`, and `SuperComputer` all implement it. `UseCommandWhistleAction` and `ActivateHeavenTokenAction` each hold a `SanctuaryTool` reference and call `tool.activateSanctuaryEffect()` as neither action imports the concrete class. `SuperComputer` also implements `SanctuaryTool`, meaning the terminal itself participates in the same abstraction as the purchasable tools.

Pros	Cons
DIP: action classes depend on <code>SanctuaryTool</code> only. Neither <code>UseCommandWhistleAction</code> nor <code>ActivateHeavenTokenAction</code> references a concrete item class.	Requires all tool items to implement a two parameter interface contract. A developer adding a new tool must understand the interface semantics to implement <code>activateSanctuaryEffect()</code> correctly.
OCP: a new <code>SanctuaryTool</code> item (e.g., a future <code>ShieldOrb</code>) requires only implementing the interface and adding a purchase option to <code>SuperComputer</code> and has zero changes to existing action classes.	
<code>SuperComputer</code> implementing <code>SanctuaryTool</code> unifies the terminal and handheld tools under a single abstraction, making the audit system extensible without special casing the ground.	
Consistent with <code>DistortionSource</code> : both interfaces follow the same "interface on ground/item, thin action delegates" pattern, making the REQ4 architecture immediately learnable.	

Final Design Justification

Design 2 was chosen. The `SanctuaryTool` interface is the direct counterpart of `DistortionSource` on the protective side: anomaly grounds implement `DistortionSource`, protective tools implement `SanctuaryTool`, and neither set of action classes ever references a concrete class (DIP). The single `activateSanctuaryEffect()` method is intentionally broad as each implementor defines its own complex effect, keeping the interface small and the implementations rich (Interface Segregation Principle). `SuperComputer` implementing `SanctuaryTool` is a deliberate design choice: it allows the terminal to participate in the same activation pattern as handheld items, meaning future systems can scan for `SanctuaryTool` implementors on the map without any special terminal logic.

M. Design of the Extinguishable Interface

Design 1: Add `extinguish()` directly to `DistortionSource`

Pros	Cons
No new interface as <code>extinguish()</code> is available on any <code>DistortionSource</code> .	SRP violated on the interface: <code>DistortionSource</code> conflates anomaly audit/stabilisation semantics with hazard removal. Not all distortion sources are fire-based hazards.
	LSP risk: <code>BlackHolePortal</code> and <code>RageGround</code> would be forced to implement <code>extinguish()</code> with empty bodies or incorrect semantics, weakening the contract.

Design 2 (Chosen): Dedicated `Extinguishable` interface with `extinguish(Location)`

`Extinguishable` defines a single method. `BlueFire` implements both `DistortionSource` and `Extinguishable`. The audit protocol (`DistortionAuditAction`) calls `loc.getGroundAs(DistortionSource.class)` for quota logic, and separately, `loc.getGroundAs(Extinguishable.class)` for any system that needs to programmatically remove a hazard. No class is forced to implement removal semantics it does not need.

Pros	Cons
ISP satisfied: <code>DistortionSource</code> and <code>Extinguishable</code> are separate concerns. Classes opt in to only the interfaces they need.	Adds a small interface file for a single method.
Any future hazard (e.g., <code>AcidPool</code>) can implement <code>Extinguishable</code> without being a <code>DistortionSource</code> .	
<code>BlueFire.extinguish()</code> is the canonical implementation, calling <code>location.setGround(originalGround)</code> as clean, testable, and reusable.	

Final Design Justification

Design 2 was chosen. The `Extinguishable` interface satisfies the Interface Segregation Principle: a fire-based hazard that can be removed is a different contract from an anomaly that can be audited. Keeping them separate means `DistortionAuditAction` and any future "firefighting" system each depend only on the interface they need, with no unwanted method obligations. `BlueFire` implementing both interfaces is additive composition, not forced coupling as each interface is independently meaningful.

N. Design of the `DamageInterceptor` Enum

Design 1: Add `PROTECTED` to the existing `Ability` or `DistortionCapability` enum

Pros	Cons
No new enum file.	Ability carries actor-level engine capabilities (WORKER, IS_FLOOR). Adding a REQ4-specific damage mitigation flag reduces cohesion.
	DistortionCapability is semantically for ground-based distortion tags. PROTECTED is an actor state, not a ground property as adding it here would mislead any developer reading the enum.
	Both enums are shared across the codebase; adding PROTECTED risks accidental misuse.

Design 2 (Chosen): Dedicated DamageInterceptor enum in game.sanctuary with a single value PROTECTED

DamageInterceptor.PROTECTED is enabled on an actor by SanctuaryStatus, SanctuaryField, and ActivateHeavenTokenAction, and read exclusively in ContractedWorker.hurt(). Its placement in game.sanctuary signals its semantic scope: it belongs to the protective half of REQ4, not to the engine's generic capability system.

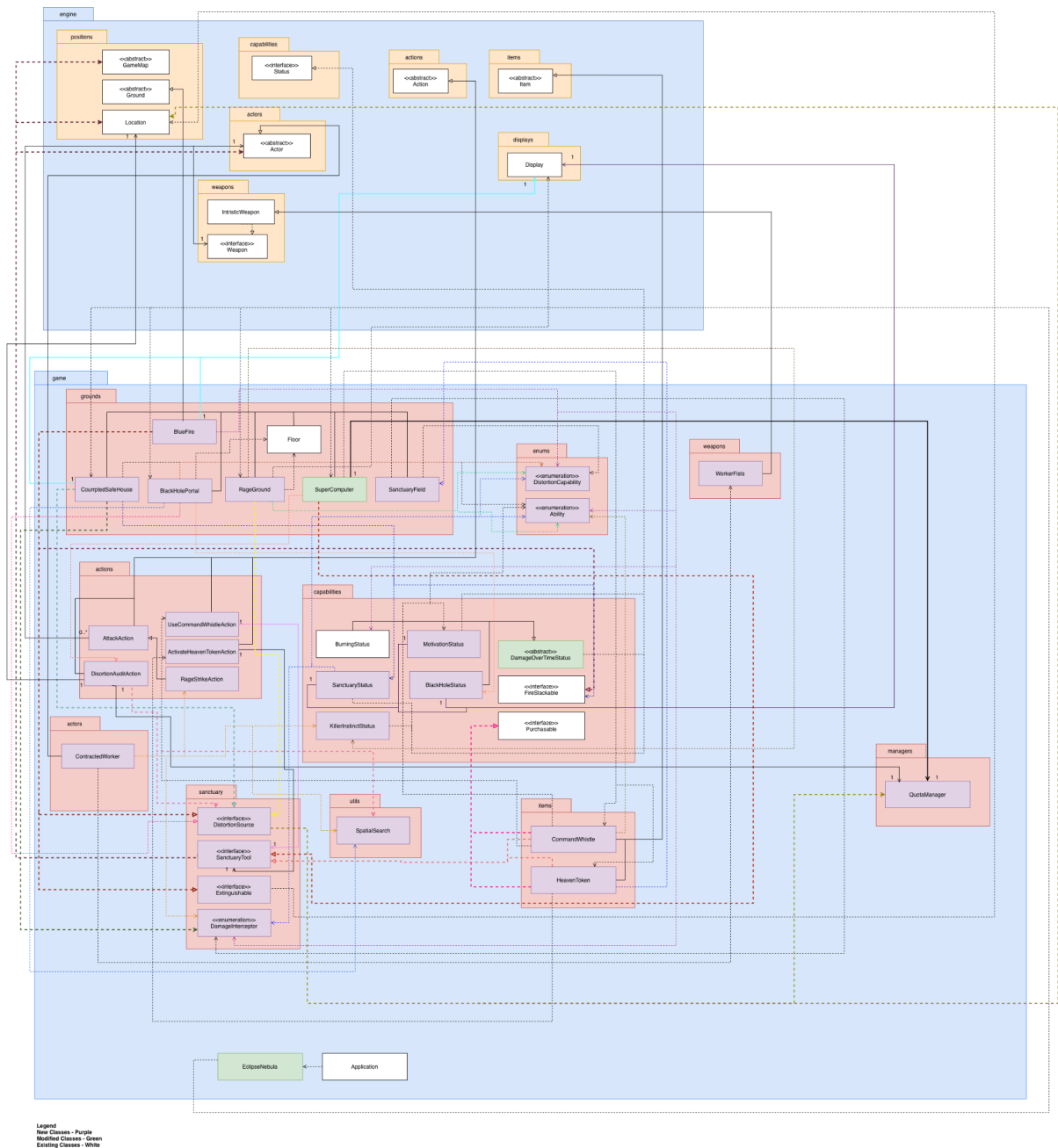
Pros	Cons
Separation of concerns: damage mitigation is namespaced independently of actor movement capabilities and ground distortion tags.	Single value enum may seem over engineered. However, future additions (e.g., DAMAGE_REDUCED, IMMUNE) extend naturally without touching other enums.
ContractedWorker.hurt() reading DamageInterceptor.PROTECTED is self documenting as the enum name makes the intent immediately clear.	
Follows the established project pattern (Ability, MaterialCapability, DistortionCapability, FacilityCapability) as consistent and immediately understandable.	
Package placement in game.sanctuary co-locates the enum with DistortionSource, SanctuaryTool, and Extinguishable, creating a coherent sanctuary package.	

Final Design Justification

Design 2 was chosen. A dedicated DamageInterceptor enum is the correct application of the separation of concerns principle. PROTECTED is a damage pipeline concept consumed by ContractedWorker.hurt() and managed by sanctuary related statuses and grounds as it has no semantic overlap with actor movement capabilities (Ability) or anomaly detection tags (DistortionCapability). Placing it in game.sanctuary alongside the other REQ4 abstractions creates a self contained package where every member has a clear, single purpose, consistent with the enum as capability pattern used throughout the codebase (DIP, ISP).

Changes from Assignment 2 for REQ4

Class	Change	Justification
SuperComputer	Implements SanctuaryTool; allowableActions() adds DistortionAuditAction and purchases for CommandWhistleand HeavenToken; QuotaManager injected via constructor	Turns the terminal into a dual-role facility node: economy hub (REQ1) and distortion sensor (REQ4), without any singleton or concrete anomaly references (DIP)
ContractedWorker	hurt() overridden to block all damage when DamageInterceptor.PROTECTED is active; playTurn() calls injectRageStrikeActions() when KillerInstinctStatus is present; isParalyzed() check added	Each addition has a single, well-scoped responsibility: protection interception in hurt(), action mutation in playTurn(), skip-turn enforcement in isParalyzed() (SRP)
SpatialSearch	Added getActorsWithinDistance(),and getLocationsWithinDistance()	Centralises REQ4 radial scan and avoids duplicating loop logic across DistortionAuditAction, RageGround, and CommandWhistle (DRY)



The diagram above shows the final design of REQ 4, which utilises the designs described in parts A through I to implement a complete Distorted Sanctuary ecosystem with capability-based anomaly detection, presence aware lifecycle management, terrain restoration via the Memento Pattern, deferred status-driven action mutation, and a REQ1 integrated corporate audit protocol.

The DistortionSource and SanctuaryTool interfaces act as a stable abstraction layer between anomaly grounds (BlackHolePortal, CorruptedSafeHouse, RageGround, BlueFire) and the protective tools (CommandWhistle, HeavenToken, SuperComputer), eliminating all cross class dependencies between concrete types (Dependency Inversion Principle). DistortionAuditAction depends exclusively on DistortionSource.audit() and never imports any ground class; UseCommandWhistleAction and ActivateHeavenTokenAction depend exclusively on SanctuaryTool.activateSanctuaryEffect() and never import CommandWhistle

or HeavenToken directly. Adding a new anomaly or tool requires only implementing the relevant interface as zero changes to any action class (Open Closed Principle).

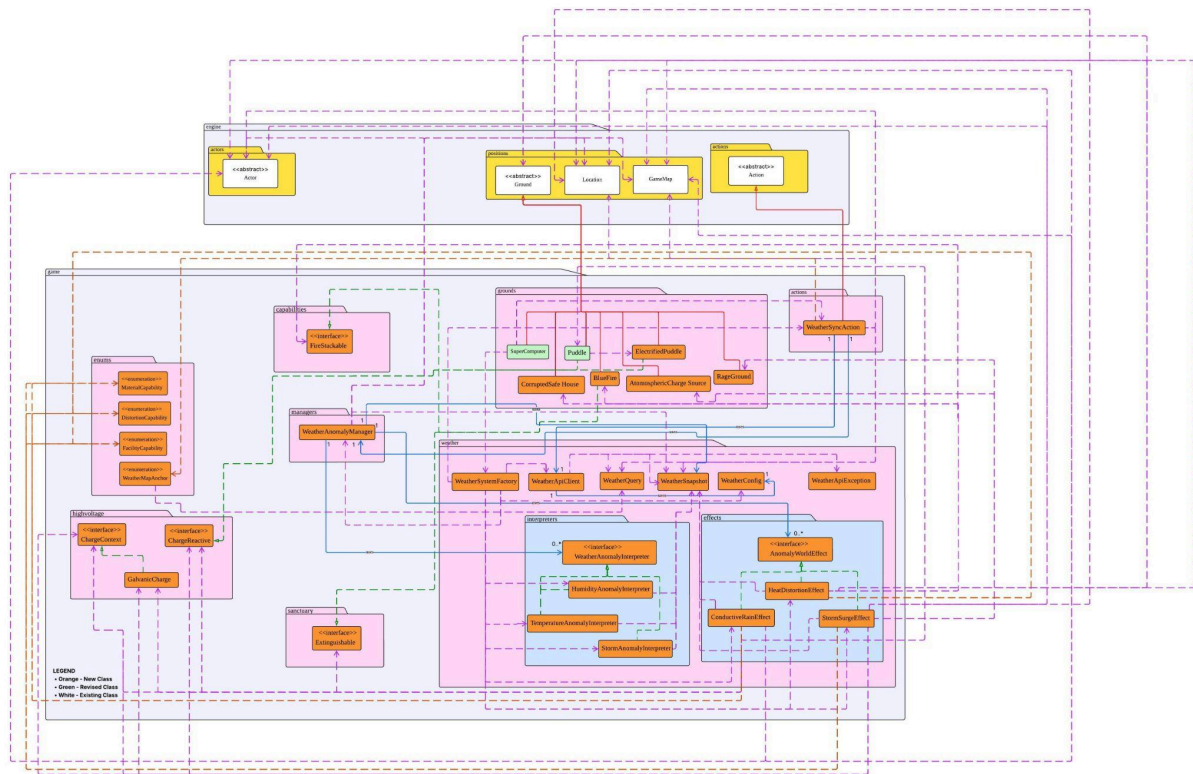
The DistortionCapability enum eliminates all instanceof checks in cross class interactions. Instead of if (ground instanceof BlueFire), the system uses ground.hasAbility(DistortionCapability.ACTIVE_HAZARD), and instead of if (ground instanceof DistortionSource), it calls loc.getGroundAs(DistortionSource.class). DamageInterceptor.PROTECTED follows the same pattern for damage mitigation: ContractedWorker.hurt() reads a capability flag rather than checking any status class directly, keeping the protection interception decoupled from any specific status implementation.

The originalGround approach, used by both BlueFire and SanctuaryField, represents a consistent pattern for replacing and later restoring terrain states: every ground that replaces another stores the original reference in its constructor and calls location.setGround(originalGround) on expiry.

This ensures any current or future ground type is faithfully restored without modifying the restoration logic (OCP). The status-driven action mutation pattern used by RageGround -> KillerInstinctStatus -> ContractedWorker.injectRageStrikeActions() respects the engine's architectural constraint that Ground.tick() cannot modify the action list directly, routing the injection through the status as a communication channel between the ground and the worker's turn processing (SRP).

Overall, REQ4 represents an architectural evolution from hard coded tick logic to a fully interface driven, capability tagged ecosystem where each class has one clear responsibility and the system is open for extension without modification.

UML DIAGRAM



DESIGN RATIONALE

REQ5: Real Weather Anomaly System

A. Weather API Communication and Query Building

WeatherApiClient, WeatherQuery, WeatherSnapshot, WeatherConfig, and WeatherApiException are used to separate the external OpenWeather API communication from the rest of the game logic. The API request is not static; it is built dynamically using game-state information such as the current map anchor and actor position.

Design 1: Implement API calling and JSON parsing directly inside WeatherSyncAction

Pros	Cons
Simple Implementation: All API logic can be placed in one action, making the first version quick to build.	Violates SRP: WeatherSyncAction would handle action execution, URL building, API communication, JSON parsing, error handling, and game-world changes.
Easy to Trace: The full weather sync process is visible inside one class.	High Coupling: WeatherSyncAction becomes directly dependent on the OpenWeather URL format and raw JSON structure.
KISS for Prototype: For a small test, fewer classes may make the feature easier to understand initially.	Weak KISS Long-Term: As the feature grows, one large action class becomes harder to read than several focused classes.
No Abstraction Overhead: No need to create separate query, snapshot, config, or exception classes.	Violates OCP: If the API provider, URL, or response format changes, WeatherSyncAction must be modified.
Direct Data Access: The raw JSON can be read immediately after the API response returns.	Weak Encapsulation: Other parts of the game may start depending on raw JSON instead of a clean weather data object.

Fast to Implement: Useful for quickly checking whether the API key and API request work.	Poor Maintainability: API failure handling becomes mixed with normal game action logic, creating a code smell.
--	--

Design 2: Separate API communication into dedicated weather classes (Current Design)

Pros	Cons
Supports SRP: WeatherApiClient handles HTTP communication, WeatherQuery builds the URL, WeatherSnapshot stores parsed values, and WeatherConfig manages the API key.	Increased Class Count: More files are required compared to putting everything inside WeatherSyncAction.
Supports OCP: If the API URL or parsing details change, the weather API classes can be updated without changing the game action flow.	Slight Indirection: Developers need to follow the flow across multiple classes instead of reading one method.
Supports Encapsulation: The rest of the game uses WeatherSnapshot instead of raw JSON strings.	Manual Parsing Limitation: Since no external JSON library is used, the parser expects specific markers in the API response.
Supports KISS in the Final Design: Each class has a small, clear purpose instead of one large class doing everything.	UML Requires More Detail: The diagram must clearly show the relationships between the support classes.
Improves Security: WeatherConfig reads OPENWEATHER_API_KEY from the environment, so the API key is not hardcoded or pushed to GitLab.	Environment Setup Required: The user must set the environment variable correctly before running the feature.

Improves Maintainability: API errors are represented with WeatherApiException, making failures easier to understand and debug.	Internet/API Dependency: The feature depends on API availability unless mocked in testing.
Reduces Coupling: WeatherSyncAction does not need to know how the API key is stored or how the HTTP request is made.	More setup is required compared to a direct hardcoded API request.

Justification: Design 2 was chosen because REQ5 requires a real API integration that meaningfully affects the game world. If all API logic was placed inside WeatherSyncAction, the action would become too large and would mix unrelated responsibilities. The final design keeps WeatherSyncAction focused on the player-triggered sync process, while WeatherApiClient handles the HTTP request and WeatherSnapshot provides a clean object containing temperature, humidity, wind speed, weather condition, city name, and country code. This follows SRP because each class has one clear role. It also improves encapsulation because the rest of the system does not depend on raw JSON. WeatherConfig was separated to protect the API key by reading it from OPENWEATHER_API_KEY instead of hardcoding it. The trade-off is that there are more classes, but this is justified because API communication, security, parsing, and game effects are separate responsibilities.

B. Weather Interpretation System

The WeatherAnomalyInterpreter interface is used so that different weather conditions can be interpreted independently. HumidityAnomalyInterpreter, TemperatureAnomalyInterpreter, and StormAnomalyInterpreter each decide whether a specific anomaly is active based on the WeatherSnapshot.

Design 1: Use conditional logic inside WeatherAnomalyManager or WeatherSyncAction

Pros	Cons
Simple Logic Flow: Humidity, temperature, wind speed, and condition checks can all be written in one method.	Violates SRP: The manager or action would be responsible for knowing every weather rule.

Fewer Classes: No interface or separate interpreter classes are needed.	Violates OCP: Adding a new anomaly such as fog or acid rain would require modifying existing manager/action code.
Easy to Debug at First: All thresholds are visible in one location.	Low Cohesion: One method would contain unrelated humidity, heat, and storm interpretation logic.
KISS for a Very Small Feature: This may be acceptable if there was only one weather condition.	Weak KISS Long-Term: As more conditions are added, the method becomes a long chain of checks.
No Polymorphism Required: The system avoids extra abstraction.	Higher Connascence of Logic: Threshold rules become tied to one central class, so changes in one weather rule risk affecting others.
Direct Control: The action/manager can immediately decide which effect should run.	High Coupling: The action/manager becomes coupled to every specific anomaly rule.

Design 2: Use WeatherAnomalyInterpreter with separate concrete interpreter classes (Current Design)

Pros	Cons
Supports SRP: Each interpreter has one responsibility, such as humidity, heat, or storm interpretation.	More Files: Each anomaly requires its own class.
Supports OCP: New weather anomalies can be added by creating a new interpreter without modifying existing interpreter classes.	Threshold Consistency Required: Interpreter thresholds must stay aligned with the matching effect thresholds.

Supports DIP: WeatherAnomalyManager depends on the WeatherAnomalyInterpreter abstraction, not concrete interpreter classes.	Slight Indirection: The manager asks interpreters whether they can respond instead of checking values directly.
Supports Polymorphism: The manager can process all interpreters through the same interface.	More Factory Setup: WeatherSystemFactory must create and provide the interpreter list.
Improves Cohesion: Humidity, temperature, and storm rules are separated into focused classes.	UML Must Be Accurate: The diagram must clearly show the three realisation relationships.
Better Maintainability: If a weather rule changes, only the relevant interpreter needs to be edited.	More class names must be maintained and kept clear.
Follows KISS at System Level: Each class remains small and easy to understand.	Slight abstraction overhead compared to a single conditional block.

Justification: Design 2 was chosen because REQ5 requires at least two new abstractions and three concrete implementations per abstraction. WeatherAnomalyInterpreter separates the decision-making step from the world-changing step. HumidityAnomalyInterpreter checks for high humidity or rain, TemperatureAnomalyInterpreter checks for high temperature, and StormAnomalyInterpreter checks for strong wind or storm conditions. WeatherAnomalyManager can loop through all interpreters without knowing their concrete types, which demonstrates Dependency Inversion. This also supports Open/Closed Principle because a new anomaly can be added through a new interpreter class instead of editing the manager. The trade-off is extra class structure, but this is appropriate because the feature has multiple weather behaviours and must remain easy to extend.

C. Weather World Effects

The `AnomalyWorldEffect` interface is used to apply the actual game-world changes caused by weather. `ConductiveRainEffect`, `HeatDistortionEffect`, and `StormSurgeEffect` each modify terrain, hazards, actors, or existing REQ3/REQ4 systems using the `WeatherSnapshot`.

Design 1: Apply all weather effects directly inside `WeatherSyncAction` or `WeatherAnomalyManager`

Pros	Cons
Direct Implementation: All terrain and actor changes can be written in one place after receiving the API response.	Violates SRP: One class would handle humidity effects, heat effects, storm effects, terrain replacement, actor damage, and hazard interaction.
Fewer Classes: No need for an <code>AnomalyWorldEffect</code> interface or separate effect classes.	Violates OCP: Adding a new effect would require modifying existing manager/action logic.
Quick Prototype: Simple weather effects can be implemented quickly.	High Coupling: The manager/action would depend directly on concrete grounds such as <code>Puddle</code> , <code>BlueFire</code> , <code>AtmosphericChargeSource</code> , and <code>RageGround</code> .
Easy to See Immediate Output: The result of each condition is written in the same method.	Poor Cohesion: One method would contain several unrelated environmental behaviours.
No Factory Setup Needed: Effects can be called directly.	DRY Risk: Similar map scanning, tile checks, and terrain protection logic may be duplicated between conditions.
KISS for a Tiny Feature: A single effect could be placed directly inside the action.	Weak KISS Long-Term: Multiple AoE effects and terrain checks would make the method large and hard to understand.

Less UML Complexity: Fewer classes and relationships need to be drawn.	Difficult Testing: Individual effects are harder to test separately if all logic is inside one method.
--	--

Design 2: Use AnomalyWorldEffect with separate concrete effect classes (Current Design)

Pros	Cons
Supports SRP: Each effect class has one environmental responsibility, such as conductive rain, heat distortion, or storm surge.	Increased Class Count: The design needs one interface and three effect classes.
Supports OCP: New weather effects can be added by implementing AnomalyWorldEffect without changing WeatherAnomalyManager.	Threshold Consistency Required: Effect thresholds must stay aligned with the related interpreter thresholds.
Supports DIP: WeatherAnomalyManager depends on AnomalyWorldEffect abstractions rather than concrete effect classes.	Slight Indirection: The manager delegates to effect objects, so developers must check the effect list to see what can run.
Supports DRY: Shared effect flow is handled by the manager, while each effect class only contains its own specific behaviour.	More setup is required in WeatherSystemFactory.
Improves Cohesion: ConductiveRainEffect, HeatDistortionEffect, and StormSurgeEffect each focus on a specific anomaly.	UML Requires More Detail: The interface and all three implementations must be shown clearly.
Enables Complex Effects: Each effect can perform AoE checks, terrain modification, hazard interaction, and actor impact.	Terrain Protection Must Be Carefully Implemented: Effects must avoid overwriting SuperComputer or important anomaly tiles.

Follows KISS at Class Level: Each effect class stays understandable instead of one giant procedural method.	More files must be maintained compared to a direct implementation.
Improves Testability: Each effect can be tested separately with controlled WeatherSnapshot values.	Slight abstraction overhead for a small prototype.

Justification: Design 2 was chosen because REQ5 is not allowed to simply print API data. It must create meaningful game-state impact. ConductiveRainEffect uses humidity and rain to create Puddle terrain, suppress Extinguishable hazards, and trigger ChargeReactive grounds through the REQ3 charge system. HeatDistortionEffect strengthens FireStackable hazards and creates BlueFire near distorted or sanctuary-related areas. StormSurgeEffect applies GalvanicCharge, triggers AtmosphericChargeSource behaviour, can damage actors, can place a new AtmosphericChargeSource, and can spread RageGround. These effects are complex because they involve area-of-effect logic, terrain modification, actor impact, and cross-requirement interaction with REQ3 and REQ4. Keeping the effects in separate AnomalyWorldEffect classes prevents WeatherAnomalyManager from becoming a large procedural class. The trade-off is that terrain protection must be handled carefully, but this is managed using capabilities such as FacilityCapability, MaterialCapability, and DistortionCapability rather than fragile concrete-class checks.

D. Weather System Construction and SuperComputer Integration

WeatherSystemFactory is used to construct the WeatherSyncAction, WeatherApiClient, WeatherAnomalyManager, interpreters, and effects. SuperComputer only offers the WeatherSyncAction to the player and does not directly create or manage the entire weather system.

Design 1: Instantiate all weather objects directly inside SuperComputer

Pros	Cons
Simple Access Point: SuperComputer can directly create the weather action and all its dependencies.	Violates SRP: SuperComputer would become responsible for facility interaction and weather system construction.
Fewer Classes: No WeatherSystemFactory is required.	High Coupling: SuperComputer would depend on WeatherApiClient, WeatherConfig,

	WeatherAnomalyManager, every interpreter, and every effect.
Easy to Follow Initially: The full object creation process is visible inside one class.	Violates OCP: Adding or replacing an effect would require modifying SuperComputer.
KISS for a Small Demo: Direct construction may be acceptable if the system only had one dependency.	Weak KISS Long-Term: SuperComputer would become harder to read as more facility actions and weather dependencies are added.
Less UML Detail: One less class needs to be shown in the diagram.	Low Cohesion: SuperComputer already handles quota/economy access, so adding weather construction gives it unrelated responsibilities.
Direct Integration: The action can be connected immediately where it is offered.	Poor Maintainability: Any weather setup change would require editing the facility class.

Design 2: Use WeatherSystemFactory to create the weather action and dependencies (Current Design)

Pros	Cons
Supports SRP: SuperComputer offers the action, while WeatherSystemFactory handles construction.	Additional Class Required: The factory adds one more class to the design.
Reduces Coupling: SuperComputer does not need to know every interpreter and effect class.	Static Factory Limitation: A static factory is simple but less flexible than full dependency injection.

Supports OCP: New interpreters or effects can be added in WeatherSystemFactory without changing SuperComputer.	UML Needs Dependency Arrows: The diagram must clearly show SuperComputer depending on WeatherSystemFactory and WeatherSyncAction.
Supports DIP: The factory injects lists of WeatherAnomalyInterpreter and AnomalyWorldEffect abstractions into WeatherAnomalyManager.	Tests may need to bypass the factory to inject fake dependencies.
Improves Cohesion: SuperComputer remains a facility class instead of becoming a weather system builder.	Developers must check WeatherSystemFactory to see the full weather setup.
Follows KISS at Integration Level: Construction is centralised in one simple factory rather than scattered across the game.	The factory must be updated whenever new interpreter or effect classes are added.
Improves Maintainability: Weather setup changes are isolated to the factory.	More setup code than direct construction.

Justification: Design 2 was chosen because SuperComputer should not be responsible for constructing the full weather API system. SuperComputer is already responsible for offering facility-related actions, so placing all API, interpreter, and effect creation inside it would reduce cohesion and increase coupling. WeatherSystemFactory keeps object construction in one place and returns a ready-to-use WeatherSyncAction. This also helps DIP because WeatherAnomalyManager receives lists of WeatherAnomalyInterpreter and AnomalyWorldEffect rather than creating concrete classes itself. The trade-off is that WeatherSystemFactory adds another class, but it keeps SuperComputer clean, maintainable, and focused on its original responsibility.

Final Design Summary

The final REQ5 design implements a Real Weather Anomaly System that connects the game to the OpenWeather API and uses real weather data to alter the game world. The system does not simply print weather information. Instead, values such as humidity, temperature, wind speed, and weather condition are interpreted into anomalies that modify terrain, trigger existing charge systems, interact with hazards, and affect actors.

The design uses two main abstractions: `WeatherAnomalyInterpreter` and `AnomalyWorldEffect`. `WeatherAnomalyInterpreter` represents the interpretation layer, where `HumidityAnomalyInterpreter`, `TemperatureAnomalyInterpreter`, and `StormAnomalyInterpreter` decide whether a particular anomaly condition is active. `AnomalyWorldEffect` represents the world-changing layer, where `ConductiveRainEffect`, `HeatDistortionEffect`, and `StormSurgeEffect` apply complex changes to the map and actors.

`WeatherAnomalyManager` demonstrates Dependency Inversion because it stores lists of `WeatherAnomalyInterpreter` and `AnomalyWorldEffect` rather than depending directly on concrete classes. This means the manager does not need to know whether the effect is humidity, heat, storm, or a future anomaly. It simply asks each abstraction whether it can respond and delegates the work. This also supports Open/Closed Principle because new weather anomalies can be added by creating new interpreter and effect classes without changing the manager.

The API support classes keep the implementation maintainable. `WeatherQuery` builds the dynamic API request using game-state information, `WeatherApiClient` performs the request, `WeatherSnapshot` stores the parsed result, `WeatherConfig` reads the API key from the `OPENWEATHER_API_KEY` environment variable, and `WeatherApiException` gives the system a clear failure type. This avoids hardcoding the API key and prevents raw JSON from leaking into the rest of the game logic.

The world effects satisfy the requirement for meaningful and complex game-state impact. `ConductiveRainEffect` creates Puddle terrain, suppresses Extinguishable hazards, and triggers ChargeReactive grounds through the REQ3 charge system. `HeatDistortionEffect` strengthens FireStackable hazards and creates BlueFire around distorted or sanctuary-related tiles. `StormSurgeEffect` applies GalvanicCharge, triggers `AtmosphericChargeSource` behaviour, can damage actors, can place a new `AtmosphericChargeSource`, and can spread RageGround. These behaviours involve terrain modification, AoE checks, actor impact, and cross-component interaction.

The design also avoids switch statements and instanceof checks by using polymorphism and capability-based checks instead of manually checking concrete class types. `FacilityCapability`, `MaterialCapability`, and `DistortionCapability` are used to decide whether tiles are safe to modify. This protects important terrain such as the SuperComputer and avoids fragile class-specific checks. It also improves maintainability because future protected terrain can be identified through capabilities rather than hardcoded concrete class checks.

Hypothetical Extension Example: If a future requirement introduced a fog anomaly, a developer could add `FogAnomalyInterpreter` and `FogWorldEffect`. `WeatherAnomalyManager` would not need to be modified because it already works through `WeatherAnomalyInterpreter`.

and AnomalyWorldEffect lists. Similarly, if the game later used a different weather API, WeatherApiClient and WeatherQuery could be updated while the rest of the system continued to use WeatherSnapshot.

Trade-off Analysis: The main trade-off in the chosen design is abstraction overhead versus long-term maintainability. REQ5 requires API communication, dynamic query construction, two abstractions, six concrete classes, higher-level classes using abstractions, and meaningful game-state impact. Because of that complexity, the extra structure is justified. The final design keeps responsibilities separated, avoids placing all logic inside SuperComputer or WeatherSyncAction, and supports DRY, SRP, OCP, DIP, KISS, encapsulation, polymorphism, low coupling, and high cohesion.